DAREB · TECHNICAL DEFENSE Q&A

درب

أسئلة وأجوبة المناقشة التقنية

كل ما يمكن أن يسأله مهندس خبير عن Flutter وDart، وعن بنية المشروع وقراراته، وعن طريقة التفكير الهندسي، مع جواب مختصر يمكن قوله بثقة، ومثال من كود «درب» نفسه.

160	20	96	44
سؤالاً وجواباً	أساسي	متوسط	متقدم

المشروع	تطبيق «درب» التعليمي: مشروع تخرج دورة تمكين (100 ساعة)
المدخلات	مجموعة Postman لواجهة Laravel، وتصميم فيجما
التقنيات	`dartz` · `dio` · `get_it` · `go_router` · `flutter_bloc` · `Dart 3` · `Flutter`
الطالب	Rashed eybo

كيف تستخدم هذا الدليل

المناقش الخبير لا يختبر الحفظ. يريد أن يعرف ثلاثة أشياء: هل تفهم ما كتبت، وهل تعرف **لماذا** اخترت هذا الحل دون غيره، وهل تعرف حدود حلك. لذلك كل جواب هنا مكتوب ليُقال بصوتك في دقيقة أو أقل، ومعه مثال من مشروعك.

طريقة الجواب في أربع خطوات

1. التعريف
جملة واحدة تقول ما هو الشيء.

2. السبب
لماذا يوجد، أي مشكلة يحل.

3. البديل
ما الخيار الآخر، ولماذا لم تختره.

4. المثال
أين استخدمته في «درب»، باسم الملف.

مثال: «ما هو `guard`؟»

1. دالة تنفّذ استدعاء مصدر البيانات وتحوّل استثناءاته إلى `Failure`.
2. حتى لا يتكرر `try/catch` في كل دالة في كل مستودع.
3. البديل أن يكتب كل مستودع `try/catch` بنفسه، فتتكرر الأخطاء وتنسى حالةً ما.
4. في `lib/core/error/guard.dart`، وكل مستودع يكتب `guard(_remote.getX)`.

مستويات الأسئلة

أساسي يُسأل لأي مطور Flutter، ويجب أن يكون الجواب فورياً.

متوسط يميّز من بنى تطبيقاً حقيقياً عمّن تابع دروساً فقط.

متقدم يسأله السينيور ليعرف عمق فهمك وطريقة تفكيرك.

إن ضاق الوقت: احفظ الأساسي كاملاً، والجزء الأول (المشروع) والثالث عشر (طريقة التفكير) كاملين، ثم المتوسط.

ثلاث قواعد يوم المناقشة

- **اربط كل جواب بمشروعك.** جملة «في درب فعلت كذا في الملف كذا» أقوى من أي تعريف محفوظ.
- **قل «لا أعرف» بثقة حين لا تعرف**، ثم قل كيف ستبحث: «لم أستخدمه، لكني أتوقع أنه... وسأتحقق من التوثيق». المناقش يحترم هذا أكثر من جواب مخترع.
- **اعرف نقاط ضعف مشروعك قبل أن يسألك عنها** (الجزء 13). من يذكر حدود حله بنفسه يبدو خبيراً لا مدافعاً.

أرقام مشروعك (احفظها)

- 13 ميزة في `lib/features`، كل واحدة بثلاث طبقات: `domain` و`data` و`presentation`.
- 23 Cubit، و338 اختباراً ناجحاً، و`flutter analyze` بلا أي ملاحظة.
- كل الميزات مربوطة بخادم Laravel الحقيقي، عدا التنزيلات التي ما زالت ببيانات تجريبية.
- 54 إطاراً من فيجما محفوظة في `docs/design` مع فهرس يربط كل إطار بمساره في التطبيق.

الفهرس

1. المشروع: قصته وقراراته — من 1 إلى 12
2. لغة Dart — من 13 إلى 29
3. أساسيات Flutter — من 30 إلى 49
4. إدارة الحالة: Bloc وCubit — من 50 إلى 62
5. Clean Architecture وبنية المشروع — من 63 إلى 79
6. الشبكة والتعامل مع الـ API — من 80 إلى 92
7. التنقل والجلسة: go_router — من 93 إلى 100
8. الواجهة ومطابقة تصميم فيجما — من 101 إلى 110
9. الاختبارات — من 111 إلى 122
10. الأداء — من 123 إلى 129
11. الأمان — من 130 إلى 136
12. طريقة العمل وGit والنشر — من 137 إلى 146
13. طريقة التفكير وأسئلة الخبراء — من 147 إلى 160
14. مراجعة سريعة: مصطلحات في سطر — آخر الدليل

الجزء 1

المشروع: قصته وقراراته

أول ما يُسأل في أي مناقشة. هنا تُثبت أنك صاحب المشروع لا ناسخه: ماذا بنيت، وكيف بدأت، ولماذا اخترت ما اخترت.

س1 عرّفنا بمشروعك في دقيقة.

أساسي

«درب» تطبيق تعليمي بـ Flutter لطلاب المدارس. فيه:

- المواد والدورات، والدروس فيديو وملفات.
- مسار تعلم متسلسل.
- اختبارات ونشاطات يومية بنتائج ومراجعة.
- اشتراكات بالكود، ومعرض صور، وإشعارات، وإنجازات، وملف شخصي.

بنيته من مدخلين فقط: **مجموعة Postman** لواجهة Laravel، و**تصميم فيجما**. المعمارية Clean Architecture لكل ميزة، وإدارة الحالة بـ Cubit، والتنقل بـ go_router، والشبكة بـ Dio. كل الميزات مربوطة بالخادم الحقيقي، وعليه 338 اختباراً.

س2 من أين بدأت؟ كيف حوّلت ملف Postman وتصميم فيجما إلى تطبيق؟

متوسط

بخطوة سمّيتها «الفهم قبل الكود»، لكل ميزة:

1. **من التصميم:** كتبت الشاشات، وما يفعله المستخدم في كل شاشة، والحالات الأربع لكل شاشة: تحميل، فارغ، خطأ، نجاح.
2. **من Postman:** أرسلت طلباً حقيقياً لكل endpoint، وسجّلت:
   - شكل `data` الفعلي.
   - الحقول التي تأتي `null` .
   - ردود الخطأ: 400، 401، 404، 422.
3. **ثم الطبقات بالترتيب:** `domain` ثم `data` ثم Cubit ثم الواجهة، مع commit بعد كل خطوة.

> **في مشروعي:** هذه الخطوات مكتوبة في `docs/FEATURE_WORKFLOW.md` ، وكل ميزة بُنيت ورُوجعت عليها.

س3 لماذا Clean Architecture؟ أليست تعقيداً زائداً لتطبيق بهذا الحجم؟

متوسط

سؤال مشروع، وجوابي أنها أفادتني فعلاً في ثلاث مواقف:

- **بدأت ببيانات تجريبية** قبل جهوزية بعض الـ endpoints. ثم ربطت كل ميزة بالخادم بتغيير سطر واحد في حقن الاعتماديات، دون لمس الـ Cubit ولا الواجهة.

- **الاختبارات:** أختبر الـ Cubit بمستودع مزيّف، وأختبر المستودع بمحوّل HTTP مزيّف، دون خادم.
- **تغيّر شكل الرد** يُصلَح في Model واحد، ولا ينتشر في الشاشات.

الثمن ملفات أكثر لكل ميزة. لكن 13 ميزة يعمل عليها أكثر من شخص (أنا وأدوات المساعدة والمراجعة) تحتاج قواعد ثابتة. لتطبيق من شاشتين لكنت اكتفيت بطبقتين.

تجنّب: لا تقل «لأنها أفضل معمارية». قل ما الذي وفّرته لك في مشروعك تحديداً.

س4 ما الحزم التي استخدمتها، ولماذا كل واحدة؟

متوسط

الحزمة	لماذا
`flutter_bloc`	إدارة الحالة بـ Cubit: حالة واضحة قابلة للاختبار، وفصل المنطق عن الواجهة.
`get_it`	حقن الاعتماديات: مكان واحد يربط المصدر بالمستودع بالـ UseCase بالـ Cubit.
`dartz`	النوع `Either<Failure, T>` : يجعل الفشل جزءاً من نوع الدالة، فلا يُنسى.
`go_router`	تنقل بمسارات نصية، وحارس للجلسة (`redirect`)، وتبويبات سفلية تحفظ حالتها.
`dio`	عميل HTTP بـ interceptors للتوكن، و `FormData` لرفع الملفات، ومهلات اتصال.
`equatable`	مقارنة الكيانات والحالات بالقيمة، فلا يُعاد البناء إن لم تتغير الحالة فعلاً.
`shared_preferences`	حفظ التوكن والمستخدم ومعرّف الجهاز.
`cached_network_image`	تخزين الصور مؤقتاً على الجهاز.
`youtube_player_iframe`	تشغيل دروس يوتيوب داخل التطبيق.
`flutter_svg` · `image_picker` · `url_launcher`	أيقونات التصميم، وصورة الملف الشخصي، وفتح الروابط والملفات.

س5 ما أصعب مشكلة واجهتها، وكيف حللتها؟

متوسط

اختر واحدة واحكها كقصة: المشكلة، ثم كيف عرفت السبب، ثم الحل، ثم كيف تأكدت.

- **تجمّد صفحة الدرس مع مشغّل يوتيوب:**
  - السبب: قصّ `WebView` بزوايا دائرية داخل صفحة قابلة للتمرير، واستدعاء `setState` أثناء إنشاء المشغّل.
  - الحل: عدم القص حول المشغّل، وتأجيل معالجة الخطأ إلى ما بعد الإطار الحالي بـ `addPostFrameCallback` .
- **ردود بحث قديمة تظهر فوق الجديدة:**
  - السبب: الطالب يكتب «ريا» ثم «رياضيات»، والرد الأول يصل متأخراً فيكتب فوق الثاني.
  - الحل: أتجاهل أي رد لا يطابق النص الحالي، وأنتظر 400ms بعد آخر حرف قبل الطلب.

- **شكل رد غير ثابت:** المشرف يكتب مدة الدرس نصاً حراً (`00:50:05` أو `7`)، فكتبت دالة تفهم الشكلين، واختبرتها.

س6 ماذا فعلت حين اختلف الخادم عن التصميم؟

متوسط

قاعدتي: **الخادم مصدر الحقيقة، والتصميم مصدر الشكل.**

- إن عرض التصميم شيئاً لا يرسله الخادم (نقاط الاختبار، عدد صفحات الملف) لا أخترع له قيمة. أخفيه، أو أعرض بديلاً صادقاً مثل «ملف PDF»، وأوثّق الفرق.
- إن أرسل الخادم شيئاً لا مكان له في التصميم، أتركه في الـ Model إن كان له استخدام قريب، وإلا لا أقرؤه.
- الأخطاء الإملائية في التصميم أكتبها صحيحة، وأذكرها.

> **في مشروعي:** مثال: نسبة `semester_rate` ظهرت 75% لمادة لم يُحضر فيها أي درس. لم أعرضها شريطاً «للوحدات المكتملة» رغم أن التصميم يطلبه، لأنها ليست نسبة إنجاز.

س7 هل اكتشفت مشكلات في الواجهة الخلفية؟ وماذا فعلت بها؟

متقدم

- **رابط الدرس المقفل يصل في الرد**، فالقفل في التطبيق شكلي فقط. أي شخص يقرأ الرد يحصل على الفيديو.
- **إجابات الاختبار الصحيحة تصل قبل الحل** في `exams/result` ، فيمكن الغش بقراءة الرد.
- **ترتيب الدروس المتسلسل** يفرضه التطبيق وحده، والخادم يقبل حضور أي درس.
- `exams/my` **يعود فارغاً** رغم الاشتراك، وبعض الأسئلة لها أكثر من إجابة صحيحة.

لم أُخفِ المشكلات ولم أصلحها بحيلة في التطبيق. وثّقتها لمطوّر الخادم، لأن **الأمان مسؤولية الخادم**: التطبيق يمكن تعديله أو تجاوزه، أما الخادم فلا.

س8 ما الميزات (الوحدات) في التطبيق؟

أساسي

13 ميزة في `lib/features` :

- `auth` : الدخول برمز التحقق، والتسجيل.
- `splash` : شاشة البداية.
- `home` : الرئيسية.
- `courses` : المواد والدورات والدروس والنقاشات و«موادي».
- `search` : البحث المجمّع.
- `quiz` : الاختبارات والنشاطات.
- `subscription` : الباقات والتفعيل بالكود.
- `gallery` : المعرض.
- `notifications` : الإشعارات.
- `achievements` : الإنجازات.
- `profile` : الملف الشخصي.
- `support` : تواصل معنا والأسئلة الشائعة والسياسات.

- `downloads` : التنزيلات.

س9 كيف يعمل «مسار التعلم» وقفل الدروس؟

متوسط

دالة نقية `learningPath` تأخذ فصول الدورة وتعيد لكل درس حالته:

- `open` : يُفتح الآن.
- `locked` : مدفوع والطالب غير مشترك.
- `waiting` : قبله في فصله درس مفتوح لم يحضره الطالب.
- `exam` : اختبار.

وتعلّم أول درس لم يُحضر بأنه «قيد التشغيل».

```
if (lesson.type == LessonType.exam)      -> exam
else if (lesson.isLocked)                -> locked
else if (pending != null)                -> waiting  // درس سابق لم يُحضر
else if (lesson.isAttended)              -> open
else { open + isCurrent; pending = lesson; }
```

لأنها دالة نقية بلا Flutter ولا شبكة، اختبرتها وحدها في `lesson_path_test.dart` . والواجهة ترسم النتيجة فقط.

س10 احكِ لنا رحلة الاختبار (Quiz) من البداية إلى النهاية.

متوسط

1. `GET /exams/result?id=` يعيد الأسئلة وخياراتها، ونص السؤال HTML أحوّله إلى نص عادي في الـ Model.
2. الطالب يختار إجابة لكل سؤال، وتُحفظ في الحالة كخريطة: معرّف السؤال إلى معرّف الخيار.
3. زر «النتيجة» معطّل حتى تُجاب كل الأسئلة. عند الضغط يرسل `POST /exams/attend-exam` بالحقل `answers_id[]` .
4. صفحة النتيجة تجلب النتيجة من الخادم من جديد بمعرّف المسار، ولا تعتمد على الحالة السابقة، فتعمل حتى لو فُتحت مباشرة.
5. صفحة المراجعة تعرض إجابة الطالب، والإجابات الصحيحة إن أخطأ.

> **في مشروعي:** وجدت خطأً منطقياً: سؤال له إجابتان صحيحتان، واختيار الثانية كان يُعرض خطأً لأن المقارنة كانت مع أول إجابة صحيحة. صار الشرط «هل الخيار المختار صحيح؟»، مع اختبار يثبت ذلك.

س11 كيف يعمل تسجيل الدخول؟

متوسط

1. `POST auth/login` برقم الهاتف، فيرسل الخادم رمز تحقق.
2. `POST auth/active` بالرقم والرمز ومعرّف الجهاز `device_id` ، فيعيد `token` وبيانات المستخدم.
3. أحفظ التوكن والمستخدم محلياً، ويضيف `ApiClient` التوكن لكل طلب.
4. الحساب الجديد يأتي بلا اسم، فيكمل التسجيل بـ `users/edit` : الاسم والمدرسة والفرع والعمر والجنس.

معرّف الجهاز عشوائي آمن يُولَّد مرة واحدة ويُحفظ (`Random.secure`)، ويبقى بعد تسجيل الخروج.

س12 كيف يشترك الطالب ويدفع؟

متوسط

لا توجد بوابة دفع في الواجهة الخلفية. الدفع يتم خارج التطبيق عند وكيل:

1. يختار الطالب باقة.
2. يرى طرق الدفع والوكلاء من `payment-methods/all` ، ويدفع للوكيل نقداً أو بالحوالة.
3. يستلم كوداً ويفعّله بـ `POST subscriptions/add` .

الكود الخاطئ يعود بـ 422 ورسالة يعرضها التطبيق كما هي.

تجنّب: لا تقل إن التطبيق «يدفع». قل إنه يعرض تعليمات الدفع ويفعّل الاشتراك بالكود، لأن الخادم مصمم هكذا.

الجزء 2

لغة Dart

الأساس الذي يُبنى عليه كل شيء. المناقش يبدأ غالباً بأسئلة اللغة ليعرف إن كنت تفهم ما تكتبه أم تنسخه.

س13 ما الفرق بين `var` و `final` و `const` ؟

أساسي

- `var` : متغير يُعاد إسناده، ونوعه يُستنتج من أول قيمة.
- `final` : يُسند مرة واحدة **وقت التشغيل**، مثل نتيجة طلب أو `DateTime.now()` .
- `const` : قيمة معروفة **وقت الترجمة**، ثابتة بعمق. نسخ `const` المتطابقة تشترك في كائن واحد في الذاكرة.

> **في مشروعي:** حقول الكيانات والحالات كلها `final` . والنصوص والألوان في `AppStrings` و `AppColors` كلها `static` `const`. وأكتب `const` قبل الويدجت الثابتة ليتجنب Flutter إعادة بنائها.

س14 ما Null Safety؟ واشرح `?` و `!` و `??` و `late` .

أساسي

في Dart 3 لا يقبل أي متغير القيمة `null` إلا إن صرّحت بذلك، فتُكتشف أخطاء null وقت الترجمة لا وقت التشغيل.

- `String?` : نوع يقبل `null` .
- `x!` : «أنا متأكد أنه ليس null». إن كان null ينهار التطبيق، لذلك أقلّل منه.
- `x ?? y` : القيمة البديلة إن كان `x` null. و `x?.y` تتوقف بهدوء إن كان `x` null.
- `late` : يُهيّأ لاحقاً قبل أول استخدام، ويرمي خطأ إن قُرئ قبل ذلك.

> **في مشروعي:** في الـ Models أقرأ الحقل الاختياري هكذا: `(json['professor'] as String?) ?? ''` .

س15 هل Dart متعددة الخيوط؟ وكيف يعمل `async/await` ؟

أساسي

كود Dart يعمل في **خيط واحد** داخل Isolate، مع **حلقة أحداث** (event loop).

- `await` لا يوقف التطبيق. يعلّق الدالة الحالية، ويعيد التحكم لحلقة الأحداث لترسم الإطارات وتستقبل اللمسات.
- حين يكتمل الـ `Future` تُستأنف الدالة من بعد `await` .
- لذلك الطلبات الشبكية لا تجمّد الواجهة: الانتظار على الشبكة لا على المعالج.
- الحساب الثقيل جداً على المعالج يجمّد الواجهة، وحله Isolate منفصل.

س16 ما الفرق بين `Future` و `Stream` ؟

متوسط

- `Future` : قيمة واحدة ستصل لاحقاً، أو خطأ. مثل نتيجة طلب HTTP.
- `Stream` : سلسلة قيم عبر الزمن، مثل الضغطات أو رسائل WebSocket أو تغيّرات حالة.

الـ Cubit في جوهره `Stream` من الحالات: كل `emit` قيمة جديدة في الـ Stream، و`BlocBuilder` يستمع إليها.

س17 ما حلقة الأحداث (Event Loop)؟ وما الفرق بين microtask وevent؟

متوسط

للـ Isolate طابوران:

- **microtask queue:** أولوية أعلى، تُفرَّغ كاملة قبل أي حدث. ومنها تكملة الـ `Future` المكتملة.
- **event queue:** المؤقتات، والإدخال والإخراج، والضغطات، والإطارات.

الحلقة تأخذ حدثاً، ثم تفرّغ كل الـ microtasks، ثم الحدث التالي. لذلك سيل microtasks لا ينتهي يجمّد الواجهة.

س18 ما الـ Isolate؟ ومتى تستخدم `compute` أو `Isolate.run`؟

متقدم

Isolate خيط تنفيذ بذاكرة منفصلة تماماً، لا يشارك المتغيرات مع غيره، ويتواصل بالرسائل فقط. لذلك لا توجد فيه مشكلات الأقفال وتعارض الخيوط.

أستخدمه حين يأخذ عمل حسابي أكثر من بضعة ملّي ثوانٍ، مثل:

- فك JSON ضخم بآلاف العناصر.
- معالجة صورة، أو تشفير.

```
final items = await Isolate.run(() => parseHugeJson(raw));
```

> **في مشروعي:** الردود صغيرة (عشرات العناصر)، ففكّها في الخيط الرئيسي لا يُلاحظ. لو جاءت قوائم ضخمة لنقلت `parse` إلى `Isolate.run`.

س19 ما الفرق بين `extends` و`implements` و`with`؟

متوسط

- `extends` : وراثة. تأخذ التنفيذ وتعدّل ما تريد. وراثة واحدة فقط.
- `implements` : تتعهد بتنفيذ الواجهة كلها بنفسك. كل صنف في Dart واجهة ضمنية.
- `with` : تمزج `mixin` ، أي سلوك جاهز يُضاف لأصناف لا علاقة وراثة بينها.

> **في مشروعي:** `CourseModel extends Course` : الـ Model يرث الكيان ويضيف `fromJson` .
> و `CoursesRepositoryImpl implements CoursesRepository` : التنفيذ يلتزم بالعقد المجرد. والمستودعات المزيّفة في الاختبارات `implements` أيضاً.

س20 كيف تعرّف واجهة (interface) في Dart؟ وما معدّلات الأصناف في Dart 3؟

متوسط

لا توجد كلمة `interface` قديماً. يُكتب `abstract class` بدوال بلا جسم، ويُنفَّذ بـ `implements` .

Dart 3 أضافت معدّلات تتحكم بطريقة استخدام الصنف:

- `interface class` : يُنفَّذ ولا يُورث من خارج مكتبته.
- `base class` : يُورث ولا يُنفَّذ.
- `final class` : لا يُورث ولا يُنفَّذ من خارج مكتبته.
- `sealed class` : أنواعه الفرعية معروفة كلها، فيفحص المترجم أن `switch` يغطيها جميعاً.

في مشروعي: `AppColors` و `AppStrings` معرّفة `abstract final class` : لا تُنشأ منها نسخ ولا تُورث، فهي مستودع ثوابت فقط.

س21 ما الـ Generics؟ أعطِ مثالاً من مشروعك.

متوسط

كود يعمل مع أي نوع مع الحفاظ على أمان الأنواع، بدل `dynamic` .

```
abstract class UseCase<T, Params> {
  Future<Either<Failure, T>> call(Params params);
}

Future<T> get<T>(String path, {T Function(dynamic data)? parse});
```

`GetMyCourses` هو `UseCase<List<Course>, NoParams>` ، فيعرف المترجم أن نتيجته قائمة دورات.

و `ApiClient.get<T>` يعيد نوع ما يرجعه `parse` .

س22 ما الـ Records والـ Pattern Matching في Dart 3؟ وهل استخدمتها؟

متوسط

- **Record:** مجموعة قيم بلا صنف، مثل `(Semester, List<PathLesson>)` .
- **Patterns:** تفكيك القيم وفحصها في سطر واحد.

```
return switch (state.status) {
  QuizStatus.failure => ErrorView(message: state.error, onRetry: ...),
  QuizStatus.success when quiz != null => _Questions(quiz: quiz),
  _ => const LoadingView(),
};

if (titleIcon case final icon?) Icon(icon);   // إن لم يكن null
for (final (i, answer) in question.answers.indexed) { ... }
```

في مشروعي: `learningPath` تعيد `List<(Semester, List<PathLesson>)>` ، وكل الصفحات تختار ما تعرضه بـ `switch` على حالة الـ Cubit.

س23 ما فائدة `sealed class` ؟ وأين كنت ستستخدمها؟

متقدم

تجعل المترجم يعرف كل الأنواع الفرعية. فإن أضفت نوعاً جديداً ولم تعالجه في `switch` ، يفشل البناء بدل أن يظهر الخطأ للمستخدم.

```
sealed class Failure { const Failure(this.message); final String message; }
final class ServerFailure extends Failure { ... }
final class AuthFailure extends Failure { ... }
```

في مشروعي: `Failure` عندي `abstract class` . تحويله إلى `sealed` تحسين جيد: الواجهة التي تتصرف بحسب نوع الفشل (مثل فتح صفحة الدخول عند `AuthFailure`) ستُجبر على تغطية كل الأنواع.

س24 ما الـ factory constructor؟ ولماذا `fromJson` مكتوب به؟

متوسط

منشئ يقرر بنفسه ما يعيده: نسخة جديدة، أو نسخة محفوظة، أو نوعاً فرعياً. وله جسم يُجري فيه حسابات قبل الإنشاء.

`fromJson` factory لأنه يقرأ الحقول ويحوّل الأنواع (`'${json['id']}'` لتصير المعرّفات نصوصاً) ثم يستدعي المنشئ العادي. والمنشئ العادي يبقى `const` بسيطاً للاختبارات.

س25 كيف تقارن كائنين في Dart؟ ولماذا تستخدم `Equatable` ؟

متوسط

`==` الافتراضي يقارن **المرجع**: كائنان بنفس القيم غير متساويين. `Equatable` يجعل المقارنة **بالقيمة** عبر `props` ، ويولّد `hashCode` متسقاً معها.

هذا مهم في Bloc لسببين:

- `emit` لحالة مساوية للحالية لا يعيد البناء.
- الاختبارات تقارن الحالة المتوقعة بـ `expect` مباشرة.

تجنّب: نسيان حقل في `props` : تغيّره لا يعيد بناء الواجهة. لذلك قاعدتي أن `props` تضم كل الحقول.

س26 ما الفرق بين المعاملات الموضعية والمسمّاة؟ ومتى تستخدم `required` ؟

أساسي

- **الموضعية:** `f(a, b)` ، ترتيبها مهم. تناسب المعامل الواحد الواضح، مثل `GetCourses(repository)` .
- **المسمّاة:** `f(a: 1, b: 2)` ، تُقرأ أوضح وترتيبها حر. وتُستخدم `required` لما لا بد منه.

في مشروعي: الـ Cubit الذي يأخذ أكثر من UseCase يأخذها مسمّاة: `LessonCubit(getLesson:, attendLesson:, addComment:, getCurrentUser:)` ، حتى لا يُخلط بينها.

س27 ما الـ extension methods؟

متوسط

إضافة دوال لنوع لا تملك كوده، دون وراثته:

```
extension DurationLabel on Duration {
  String get mmss => '${inMinutes}:${(inSeconds % 60).toString().padLeft(2, '0')}';
}
```

مفيدة للمساعدات الصغيرة. لكن الإفراط فيها يجعل مصدر الدالة غامضاً على القارئ، لذلك فضّلت دوالاً مسمّاة واضحة مثل `lessonDurationLabel` و `HtmlText.plain` .

س28 ما الـ `typedef` ؟ وكيف تمرّر دالة كمعامل؟

متقدم

الدوال في Dart قيم كأي قيمة: تُخزّن وتُمرَّر. و `typedef` اسم مقروء لنوع دالة.

```
typedef TokenReader = String? Function();

ApiClient({required TokenReader readToken, ...})
```

في مشروعي: `ApiClient` لا يعرف أين يُحفظ التوكن. يأخذ دالة تقرؤه، فيبقى `core` مستقلاً عن ميزة `auth` ، وأمرّر في الاختبار `() => '30|abc'` .

س29 لماذا تكتب `const` قبل الويدجت؟

متوسط

الويدجت `const` تُنشأ مرة واحدة وقت الترجمة، وكل استخدام لها هو الكائن نفسه. حين يعيد الأب البناء، يرى Flutter أن الويدجت هي هي فيتخطى إعادة بنائها هي وأبنائها. النتيجة تخصيص ذاكرة أقل وعمل أقل في كل إطار.

تجنّب: `const` لا يصلح إن كانت أي قيمة داخل الويدجت تُحسب وقت التشغيل. والقاعدة `prefer_const_constructors` ليست ضمن `flutter_lints 6` الافتراضية، فيمكن تفعيلها في `analysis_options.yaml` لتنبهك.

الجزء 3

أساسيات Flutter

كيف يرسم Flutter الشاشة، ومتى يعيد البناء، وكيف تتوزع المقاسات. أسئلة هذا الجزء تكشف بسرعة من يفهم الإطار ومن يحفظ ويدجتات.

س30 ما الويدجت (Widget)؟ وماذا تعني «كل شيء في Flutter ويدجت»؟

أساسي

الويدجت **وصف ثابت (immutable) لجزء من الواجهة**: إعدادات خفيفة، لا الشيء المرسوم نفسه. النص والهامش والمحاذاة والإيماءات والسمة كلها ويدجتات تُركّب بعضها داخل بعض (composition).

لأنها رخيصة وثابتة، يعيد Flutter إنشاءها بكثرة في كل بناء، ثم يقارنها بالسابقة ويحدّث فقط ما تغيّر.

س31 ما الأشجار الثلاث في Flutter: Widget وElement وRenderObject؟

متوسط

الشجرة	ما هي	عمرها
Widget	الوصف (الإعدادات)	قصير، يُعاد إنشاؤه مع كل بناء
Element	النسخة الحية في الشجرة، تربط الويدجت بمكانه، وتحمل `State` وتدير دورة الحياة	طويل، يبقى ما دام النوع والمفتاح نفسيهما
RenderObject	يحسب المقاسات (layout) ويرسم (paint)	طويل ومكلف، يُحدَّث ولا يُعاد إنشاؤه

عند إعادة البناء يقارن الـ Element الويدجت الجديدة بالقديمة. إن تطابق **النوع والمفتاح** يحدّث نفسه ويحافظ على الـ `State`، وإلا يهدمها وينشئ جديدة. هذا ما يجعل Flutter سريعاً رغم كثرة إعادة البناء.

س32 متى تستخدم StatelessWidget ومتى StatefulWidget؟

أساسي

- **Stateless:** تعتمد كلياً على مدخلاتها وعلى ما تقرؤه من السياق، مثل بطاقة الدورة `ContentCard`.
- **Stateful:** لها حالة **محلية** تتغير وتعيش مع الويدجت، مثل نص حقل البحث، والتبويب المختار، و`TextEditingController`، ومؤقت.

> **في مشروعي:** في «موادي» التبويب المختار ونص البحث حالة عرض محلية (`setState`)، أما البيانات القادمة من الخادم ففي الـ Cubit. القاعدة: ما يهم شاشة واحدة ولا يحتاج اختباراً منطقياً يبقى محلياً، وما فيه منطق أو بيانات يذهب للـ Cubit.

س33 اشرح دورة حياة StatefulWidget.

متوسط

1. `createState`: إنشاء الـ State.
2. `initState`: مرة واحدة. تهيئة المتحكمات والاشتراكات. لا يُقرأ فيها `InheritedWidget` بعد.

3. `didChangeDependencies` : بعدها، وكلما تغيّر `InheritedWidget` تعتمد عليه.
4. `build` : يتكرر كثيراً، ويجب أن يكون سريعاً وبلا آثار جانبية.
5. `didUpdateWidget` : حين يعيد الأب البناء بإعدادات جديدة للويدجت نفسها.
6. `setState` : يعلّم الـ Element بأنه «متسخ»، فيُعاد بناؤه في الإطار التالي.
7. `deactivate` ثم `dispose` : التنظيف. أغلق المتحكمات وألغِ المؤقتات والاشتراكات.

> **تجنّب:** استدعاء `setState` بعد `dispose` . لذلك أفحص `mounted` في أي ردّ متأخر، كما في مشغّل يوتيوب: `if (mounted) widget.onError();`.

س34 ما الـ `BuildContext` ؟

أساسي

هو الـ Element نفسه، أي **مكان الويدجت في الشجرة**. به نصعد في الشجرة لنجد ما فوقنا:

- `Theme.of(context)` و `MediaQuery.of(context)` .
- `context.read<Cubit>()` ، و `context.push` من go_router.

> **تجنّب:** استخدام `context` بعد `await` دون فحص `mounted` : قد تكون الصفحة أُغلقت. واستخدام سياق أعلى من الـ `BlocProvider` فلا يجد الـ Cubit.

س35 ما الـ Keys؟ ومتى تحتاجها؟

متوسط

المفتاح يقول لـ Flutter «هذه الويدجت هي نفسها، أو ليست نفسها»، حين لا يكفي النوع والمكان. نحتاجه في حالتين:

- **قوائم يتغير ترتيبها** وعناصرها Stateful: بلا مفتاح تنتقل الحالة إلى العنصر الخطأ.
- **إجبار إعادة إنشاء الحالة** حين تتغير الهوية.

أنواعه:

- `ValueKey` : بقيمة.
- `ObjectKey` : بكائن.
- `UniqueKey` : جديد دائماً.
- `GlobalKey` : فريد في التطبيق كله، يصل إلى الحالة من الخارج، وهو مكلف.

> **في مشروعي:** صفحة الدرس في الموجّه لها `ValueKey(lessonId)` ، ومشغّل يوتيوب له `ValueKey(videoId)` . عند الانتقال لدرس آخر يُنشأ Cubit ومشغّل جديدان، بدل أن يبقى فيديو الدرس السابق.

س36 اشرح قاعدة التخطيط: «القيود تنزل، والمقاسات تصعد، والأب يحدد الموضع».

متوسط

1. الأب يعطي ابنه **قيوداً**: حد أدنى وأعلى للعرض والارتفاع (`BoxConstraints`).
2. الابن يختار **مقاسه** ضمن هذه القيود ويعيده.
3. الأب يضع الابن في **موضع** ما. الابن لا يعرف موضعه.

من هنا تأتي أشهر الأخطاء:

- `ListView` داخل `Column` بلا `Expanded` : ارتفاع غير محدود.
- نص طويل في `Row` بلا `Expanded` : *RenderFlex overflowed*.

> **في مشروعي:** في صفحة النتيجة كانت بطاقة العدّاد بارتفاع ثابت 180، ففاض المحتوى حين التفّ النص على سطرين. الحل: حد أدنى للارتفاع بدل الثابت (`BoxConstraints(minHeight: 180)`)، و `IntrinsicHeight` لتتساوى البطاقات الثلاث.

س37 ما الفرق بين `Expanded` و `Flexible` ؟

أساسي

- `Expanded` : يجبر الابن على ملء نصيبه من المساحة المتبقية في `Row` أو `Column` .
- `Flexible` : يسمح للابن أن يكون أصغر من نصيبه (`FlexFit.loose`).

> **في مشروعي:** عنوان البطاقة `Flexible` مع `TextOverflow.ellipsis` ، فيأخذ عرض نصه فقط ويُقص بنقاط إن طال، وتبقى الأيقونة بعده ملاصقة له.

س38 ما الفرق بين `ListView` و `ListView.builder` و `Column` داخل `SingleChildScrollView` ؟

متوسط

- `Column` داخل `SingleChildScrollView` : يبني **كل** العناصر دفعة واحدة. مناسب لمحتوى قصير ثابت.
- `ListView(children: [...])` : يبني العناصر كلها أيضاً، لكنه قابل للتمرير بكفاءة أكبر.
- `ListView.builder` و `.separated` : **كسول**، يبني ما يظهر على الشاشة فقط. ضروري للقوائم الطويلة.

> **في مشروعي:** قوائم الدورات والاختبارات والإشعارات `ListView.separated` . وصفحة النتيجة الثابتة `SingleChildScrollView` ، حتى لا يفيض محتواها على الشاشات الصغيرة.

س39 ما الـ Slivers؟ ومتى تحتاج `CustomScrollView` ؟

متقدم

الـ Sliver جزء من منطقة تمرير يعرف كيف يتصرف أثناء التمرير. `CustomScrollView` يجمع عدة Slivers في تمرير واحد، مثل:

- ترويسة تنكمش (`SliverAppBar`).
- شبكة ثم قائمة في التمرير نفسه.
- ترويسة ثابتة لكل قسم.

كله مع البناء الكسول. أحتاجه حين تتعدد أنواع المحتوى القابل للتمرير في الصفحة نفسها. للقائمة الواحدة يكفي `ListView` .

س40 ما الفرق بين Hot Reload وHot Restart؟

أساسي

- **Hot Reload:** يحقن الكود المعدّل في التطبيق العامل ويعيد البناء، **ويحفظ الحالة**. لا يعيد تشغيل `main` ولا `initState` .
- **Hot Restart:** يعيد التطبيق من `main` وتضيع الحالة، مع أنه أسرع من التشغيل الكامل.
- **إعادة تشغيل كاملة:** عند تغيير كود أصلي (Android أو iOS)، أو إضافة حزمة بها كود أصلي، أو أصول جديدة أحياناً.

تجنّب: لو غيّرت تهيئة في `initDependencies` أو في `main` لن يظهر أثرها بـ Hot Reload.

س41 ما الـ `InheritedWidget`؟ وما علاقته بـ `Theme.of` و`BlocProvider`؟

متوسط

ويدجت تجعل بيانات متاحة لكل من تحتها في الشجرة، بسرعة ودون تمريرها يدوياً. والمعتمد عليها يُعاد بناؤه تلقائياً حين تتغير.

هي أساس `Theme.of` و`MediaQuery.of` و`Directionality`. و`BlocProvider` مبني على حزمة `provider` التي تبني على `InheritedWidget`، لذلك `context.read<MyCubit>()` يجد أقرب Cubit فوقه.

س42 كيف جعلت الواجهة تعمل على مقاسات الشاشات المختلفة؟

متوسط

- التصميم بعرض 430، لكني لا أكتب عروضاً مطلقة للحاويات الكبيرة. أستخدم `Expanded` والنسب والهوامش، فتتمدد على الشاشات العريضة وتنضغط على الضيقة.
- المحتوى الطويل داخل منطقة تمرير، حتى لا يفيض على الشاشات القصيرة.
- `SafeArea` و`MediaQuery.paddingOf(context)` لتجنّب النتوء وشريط الإيماءات.
- النصوص بلا ارتفاعات ثابتة، حتى تتحمل تكبير الخط من إعدادات الجهاز.

س43 التطبيق عربي. كيف تعاملت مع الاتجاه من اليمين إلى اليسار (RTL)؟

متوسط

- اللغة `Locale('ar')` مع `GlobalMaterialLocalizations`، فيصير الاتجاه RTL تلقائياً.
- لا أكتب `left` و`right` في التخطيط. أكتب **start** و**end**: `EdgeInsetsDirectional` و`AlignmentDirectional` و`PositionedDirectional`. فلو أُضيفت الإنجليزية لاحقاً ينقلب كل شيء صحيحاً.
- الأيقونات الاتجاهية مثل السهم تُختار بحسب RTL. والتدرج اللوني في بطاقة السؤال يبدأ من `AlignmentDirectional.centerStart`.

تجنّب: استثناء مقصود: زخارف النجوم في صفحة النتيجة بمواضع `Positioned` عادية، لأنها زخرفة بصرية لا معنى لاتجاهها.

س44 كيف نظّمت السمة (Theme) والخطوط؟

متوسط

- `AppTheme.light()` يضبط `fontFamily: NotoSansArabic` مع Lato احتياطياً للحروف اللاتينية والأرقام، والألوان، وأنماط الأزرار والحقول.
- كل الألوان في `AppColors` بأسماء تصف مكانها، والنصوص في `AppStrings`، والأيقونات في `app_icons.dart`. ممنوع `Color(0x...)` أو نص عربي مكتوب مباشرة داخل الواجهة.

في مشروعي: خطأ اكتشفته: حين أعطيت زراً `ButtonStyle.textStyle` خاصاً، استبدل نمط الثيم كاملاً فضاعت عائلة الخط. الحل أن أذكر `fontFamily` في النمط الجديد صراحة. الدرس: بعض الأنماط تُستبدل كاملة ولا تُدمج.

س45 ماذا يحدث فعلاً حين تستدعي `setState`؟

متوسط

1. تُنفَّذ الدالة التي مررتها، وهي تغيّر الحقول.

2. يُعلَّم الـ Element بأنه بحاجة لإعادة بناء.
3. في الإطار التالي يُستدعى `build` لهذه الويدجت وأبنائها، ويحدّث Flutter ما تغيّر فقط.

لذلك أضع `setState` في أصغر ويدجت ممكنة، لأن كل ما تحتها يُعاد بناؤه.

س46 كيف يرسم Flutter إطاراً واحداً؟ وما ميزانية الإطار؟

متقدم

كل إطار يمر بمراحل:

1. **build:** الويدجتات.
2. **layout:** القيود والمقاسات.
3. **paint:** أوامر الرسم.
4. **composite:** الطبقات.
5. **raster:** البكسلات، بمحرك Impeller (أو Skia في الإصدارات الأقدم).

في شاشة 60Hz الميزانية نحو **16ms** للإطار، وفي 120Hz نحو 8ms. إن تجاوزها البناء أو الرسم يحدث تقطّع (jank). لذلك يبقى `build` خفيفاً، ولا يُوضع فيه عمل ثقيل ولا طلبات شبكة.

س47 ما الـ Platform Channels؟ وأين استُخدمت في مشروعك؟

متوسط

قناة رسائل بين Dart والكود الأصلي (Kotlin أو Swift) لما لا يقدر عليه Dart وحده:

- `url_launcher` : فتح المتصفح والروابط.
- `image_picker` : المعرض والكاميرا.
- `shared_preferences` : التخزين.
- `youtube_player_iframe` : WebView.

لهذا تحتاج هذه الحزم إعادة تشغيل كاملة بعد إضافتها. وفي اختبارات الواجهة لا يوجد كود أصلي، فأستبدل القناة بمعالج وهمي، كما فعلت مع الحافظة: `setMockMethodCallHandler(SystemChannels.platform, ...)` .

س48 ما الفرق بين أوضاع debug وprofile وrelease؟

متوسط

الوضع	الترجمة	الاستخدام
debug	JIT	Hot Reload والتحقق من الأخطاء (asserts). أبطأ، فلا يُقاس الأداء عليه.
profile	AOT مع أدوات القياس	قياس الأداء على جهاز حقيقي.
release	AOT محسّن	النسخة النهائية: أسرع وأصغر، وبلا أدوات تصحيح.

> **في مشروعي:** بنيت نسخة release بـ `flutter build apk --release` ، ولاحظت أن خطوط الأيقونات قُلّصت 96% تلقائياً (icon tree-shaking)، لأن Flutter يبقي فقط الأيقونات المستخدمة.

س49 ما الفرق بين `GestureDetector` و `InkWell` ؟

أساسي

- `GestureDetector` : يلتقط الإيماءات بلا أي أثر بصري، وفيه إيماءات كثيرة (سحب، تكبير، ضغط مطوّل).

- `InkWell` : ضغط مع **تموّج Material**. يحتاج سلفاً من نوع `Material` ليظهر التموّج.

> **في مشروعي:** البطاقات القابلة للضغط عندي `Material` ثم `InkWell` بالشكل نفسه (`customBorder`)، فيتبع التموّج زوايا البطاقة الدائرية.

الجزء 4

إدارة الحالة: Bloc وCubit

أكثر جزء يُسأل عنه في مقابلات Flutter. المطلوب أن تشرح لماذا اخترت Cubit، وكيف تتدفق الحالة، وكيف تتجنب أخطاءها المعروفة.

س50 ما «إدارة الحالة»؟ ولماذا لا يكفي `setState`؟

أساسي

الحالة هي كل بيانات تتغير وتؤثر في الواجهة: قائمة الدورات، وهل نحمّل، ورسالة الخطأ.

`setState` ممتاز لحالة صغيرة داخل ويدجت واحدة. لكنه عند الكبر:

- يخلط المنطق (طلبات، تحقق، أخطاء) بكود الرسم.
- يصعّب مشاركة الحالة بين شاشات.
- يمنع اختبار المنطق دون واجهة.

إدارة الحالة تفصل «ماذا يحدث» عن «كيف يُعرض».

س51 لماذا اخترت Bloc/Cubit وليس Provider أو Riverpod أو GetX؟

متوسط

- **تدفق واحد واضح:** الواجهة تستدعي دالة، والـ Cubit يصدر حالة جديدة، والواجهة ترسمها. لا تعديل للحالة من أي مكان آخر.
- **قابل للاختبار:** الـ Cubit صنف Dart عادي. أعطيه مستودعاً مزيّفاً وأفحص تسلسل الحالات.
- **معروف ومستقر** في الفرق، وتوثيقه قوي.

Riverpod خيار ممتاز أيضاً، ويقدّم أماناً أكبر وقت الترجمة ولا يعتمد على `BuildContext`. لم أختره لأن Bloc كان أنسب لما تعلمته في الدورة، ولأن قواعد فصل الطبقات عندي تنطبق عليه طبيعياً. أما GetX فيخلط التنقل والحقن والحالة في حزمة واحدة بسحر خفي يصعب اختباره.

> **تجنّب:** لا تهاجم الأدوات الأخرى. قل لماذا ناسبك اختيارك، واذكر ميزة البديل بإنصاف.

س52 ما الفرق بين Cubit وBloc؟ ومتى تختار Bloc؟

متوسط

- **Cubit:** دوال عادية (`load()` و`select()`) تصدر حالات. أبسط وأقل كوداً.
- **Bloc:** يستقبل أحداثاً (Events) يحوّلها إلى حالات. يضيف تتبعاً لكل حدث، و**محوّلات أحداث** (transformers): تجاهل التكرار، أو إلغاء السابق (`restartable`)، أو التأخير (`debounce`).

أختار Bloc حين يكون توقيت الأحداث نفسه جزءاً من المنطق، وحين أريد سجلاً لكل ما فعله المستخدم.

في مشروعي: كل الشاشات (23 Cubit) Cubit. كانت الاختبارات مكتوبة بـ Bloc وأحداث في البداية، فحوّلتها إلى Cubit لتطابق بقية المشروع. والبحث يحتاج تأخيراً، فطبّقته في الـ Cubit بـ `Timer` ، ومع Bloc كنت سأستخدم محوّل أحداث.

س53 كيف تصمم الـ State؟ اشرح بنية حالة نموذجية في مشروعك.

متوسط

```
enum NotificationsStatus { initial, loading, success, failure }

class NotificationsState extends Equatable {
  const NotificationsState({this.status = NotificationsStatus.initial,
                            this.items = const [], this.error});
  final NotificationsStatus status;
  final List<AppNotification> items;
  final String? error;            // يُعرض فقط مع failure

  NotificationsState copyWith({NotificationsStatus? status,
                               List<AppNotification>? items, String? error}) =>
      NotificationsState(status: status ?? this.status,
                         items: items ?? this.items, error: error);
  @override
  List<Object?> get props => [status, items, error];
}
```

- **ثابتة (immutable):** كل تغيير حالة جديدة بـ `copyWith` .
- **enum للحالة** بدل عدة متغيرات منطقية (`isLoading` و `hasError`)، حتى لا تجتمع حالتان متناقضتان.
- `error` في `copyWith` لا يرث القيمة القديمة عمداً، فيُمسح الخطأ تلقائياً مع أي حالة جديدة.

س54 ما الفرق بين `BlocBuilder` و `BlocListener` و `BlocConsumer` ؟

متوسط

- `BlocBuilder` : يعيد **رسم** الواجهة مع كل حالة. يُستخدم للعرض.
- `BlocListener` : ينفّذ **فعلاً مرة واحدة** لكل تغيّر: تنقل، أو SnackBar، أو حوار. لا يرسم شيئاً.
- `BlocConsumer` : الاثنان معاً.

و `buildWhen` و `listenWhen` تحددان متى يُستدعى كل منهما.

```
BlocConsumer<QuizCubit, QuizState>(
  listenWhen: (prev, curr) => prev.attend != curr.attend,
  listener: (context, state) {
    if (state.attend == AttendStatus.done) context.go('/quiz/$id/result');
    if (state.attend == AttendStatus.failure) showSnackBar(...);
  },
  builder: (context, state) => ...,
)
```

تجنّب: وضع التنقل أو SnackBar داخل `builder` : يتكرر مع كل إعادة بناء.

س55 ما الفرق بين `context.read` و `context.watch` و `context.select` ؟

متوسط

- `read` : يأخذ الـ Cubit مرة واحدة بلا اشتراك. يُستخدم **داخل دوال الضغط**: `onRetry: context.read<X>().load`.
- `watch` : يشترك ويعيد البناء مع **كل** حالة. يُستخدم في `build` .
- `select` : يشترك في **جزء واحد** من الحالة، ويعيد البناء فقط حين يتغير هذا الجزء.

```
final title = context.select(
    (SubjectDetailsCubit c) => c.state.details?.course.subjectName) ?? '';
```

في مشروعي: ترويسة صفحة المادة تعرض اسم المادة فقط، فهي تستخدم `select` ولا يُعاد بناؤها مع تبديل التبويبات أو تحديث القائمة.

تجنّب: `context.watch` داخل دالة ضغط ترمي خطأ. و `read` داخل `build` لن يعيد البناء حين تتغير الحالة.

س56 لماذا تكتب `if (isClosed) return;` بعد كل `await` في الـ Cubit؟

متقدم

لأن المستخدم قد يغلق الصفحة أثناء انتظار الرد. حينها يُغلق الـ `BlocProvider` الـ Cubit، وأي `emit` بعد الإغلاق يرمي `StateError: Cannot emit new states after calling close` .

```
Future<void> load() async {
  emit(state.copyWith(status: Status.loading));
  final result = await _getCourses(const NoParams());
  if (isClosed) return;            // الصفحة أُغلقت أثناء الطلب
  emit(result.fold(...));
}
```

في مشروعي: هذه قاعدة مكتوبة في منهج المشروع، وتُفحص في كل مراجعة.

س57 لماذا تفصل حالة «الإرسال» عن حالة «الجلب» في الـ Cubit نفسه؟

متوسط

لأن فشل الإرسال يجب ألا يمسح البيانات المعروضة. مثال صفحة الاختبار:

- الأسئلة محمّلة بنجاح (`status: success`).
- الطالب يضغط «النتيجة» فيفشل الطلب.

لو كانت حالة واحدة لصار `status: failure` واختفت الأسئلة وإجاباته. لذلك للإرسال حقل منفصل `attend: {idle, sending, done, failure}` ورسالته `attendError` : تظهر SnackBar وتبقى الأسئلة كما هي.

في مشروعي: النمط نفسه في إضافة تعليق (`commentStatus`)، وتفعيل كود الاشتراك (`activation`)، وحفظ الملف الشخصي (`saveStatus`).

س58 كيف تعاملت مع حالات السباق (Race Conditions) في البحث؟

متقدم

مشكلتان:

1. **طلب مع كل حرف.** الحل **debounce**: كل حرف يلغي المؤقت السابق، والطلب يُرسل بعد 400ms من آخر حرف.
2. **رد قديم يصل بعد رد جديد** فيكتب فوقه. الحل أن أحفظ النص في الحالة، وأتجاهل أي رد لا يطابق النص الحالي.

```
final result = await _search(query);
if (isClosed || state.query != query) return;   // رد لنص قديم
```

مع Bloc كنت سأستخدم `restartable()` من `bloc_concurrency` ليلغي معالجة الحدث السابق تلقائياً. وأُلغي المؤقت في `close()` حتى لا يعمل بعد إغلاق الصفحة.

س59 أين يُنشأ الـ Cubit؟ ومن يغلقه؟ ولماذا يُسجّل `Factory` لا Singleton؟

متوسط

يُنشأ في الموجّه عند فتح المسار، ويغلقه `BlocProvider` تلقائياً عند إغلاق الصفحة:

```
GoRoute(path: '/notifications',
  builder: (_, __) => BlocProvider(
    create: (_) => sl<NotificationsCubit>()..load(),
    child: const NotificationsPage()));
```

يُسجّل في `get_it` بـ `registerFactory` لأنه يحمل حالة شاشة: كل فتح للصفحة يحتاج نسخة نظيفة. لو كان Singleton لبقيت حالة الزيارة السابقة، ولرمى خطأ لأنه أُغلق مع الصفحة الأولى. أما المستودعات والـ UseCases فبلا حالة، فتُسجّل `LazySingleton` : نسخة واحدة تُنشأ عند أول طلب.

س60 كيف تتواصل شاشتان؟ مثلاً: حضر الطالب درساً، فكيف تتحدث صفحة الدورة؟

متقدم

اخترت أبسط حل: صفحة الدورة تنتظر نتيجة الانتقال، ثم تحدّث بياناتها بهدوء.

```
await context.push('/courses/$id/video/${lesson.id}');
if (mounted) context.read<CourseDetailsCubit>().refresh(id);
```

`refresh` لا يُظهر مؤشر تحميل، ويتجاهل الفشل حتى لا يمسح ما يراه الطالب.

البدائل حين يكثر التواصل:

- Cubit مشترك فوق الشاشتين.
- مستودع يعرض `Stream` للتغيرات.
- ناقل أحداث (event bus).

كلها تضيف ترابطاً، ولم يحتجها المشروع.

س61 ما الحالة التي تبقيها في `setState` وما التي تضعها في الـ Cubit؟

متوسط

- **في الـ Cubit:** بيانات من الخادم، وحالة تحميل أو خطأ، وأي منطق يستحق اختباراً.
- **محلية:** التبويب المختار، ونص البحث للتصفية المحلية، وفتح وطي قسم، ومتحكم نص. حالة عرض لا يحتاجها غير الشاشة.

في مشروعي: في صفحة المادة التبويب ونص البحث محليان، والتصفية تُطبَّق على قائمة الـ Cubit المحمّلة، فلا تُرسل طلبات جديدة.

س62 ما الـ Optimistic Update؟ وهل استخدمته؟

متقدم

تحديث الواجهة **قبل** وصول رد الخادم، كأن الطلب نجح، ثم التراجع إن فشل. يجعل التطبيق يبدو أسرع، كزر إعجاب يتلوّن فوراً.

لم أستخدمه لأن أفعال المشروع حساسة: حضور اختبار، أو تفعيل اشتراك، أو حفظ بيانات. هنا يجب أن يعرف الطالب النتيجة الحقيقية. أعرض حالة «يرسل» ثم النتيجة الفعلية. لو أضفنا إعجاباً بالتعليقات لاستخدمته.

الجزء 5

Clean Architecture وبنية المشروع

قلب المناقشة مع السينيور. لا يكفي أن تسمّي الطبقات: اشرح لماذا يتجه الاعتماد إلى الداخل، وما ثمن كل قرار، وأين خالفت القاعدة ولماذا.

س63 اشرح Clean Architecture كما طبقتها في مشروعك.

أساسي

كل ميزة ثلاث طبقات، والاعتماد يتجه **إلى الداخل فقط**:

```
presentation  -->  domain  <--  data
(Page, Cubit)      (Entity, Repository abstract, UseCase)      (Model, DataSource,
RepositoryImpl)
```

- **domain:** قلب الميزة، Dart خالص. لا يعرف Flutter ولا JSON ولا الخادم.
- **data:** يعرف الخادم. يحوّل الردود إلى كيانات، والاستثناءات إلى `Failure` .
- **presentation:** الصفحات والـ Cubits. لا يعرف إلا الـ UseCases.

الطلب يمر: الصفحة ← Cubit ← UseCase ← Repository ← DataSource ← `ApiClient` . والرد يعود: JSON ← `Either<Failure, Entity>` ← Model ← الواجهة.

س64 لماذا يجب ألا يعرف الـ domain شيئاً عن Flutter أو JSON؟

متوسط

لأنه أكثر جزء ثابت في التطبيق: قواعد العمل، مثل «الدرس المقفل لا يُفتح» و«النتيجة نسبة الصحيح من الكل». أما التفاصيل فتتغير: شكل الـ JSON، ومكتبة HTTP، وإطار الواجهة.

حين يعتمد القلب على التفاصيل، كل تغيير في التفاصيل يكسر القلب. وحين يكون Dart خالصاً:

- يُختبر بلا محاكي ولا شبكة.
- يمكن نظرياً نقله لواجهة أخرى (ويب، سطح مكتب، CLI) كما هو.

س65 ما الفرق بين Entity وModel؟ ولماذا يرث الـ Model الـ Entity؟

متوسط

- **Entity** (في `domain`): البيانات كما يفهمها التطبيق. أسماء واضحة، وأنواع صحيحة، والمعرّفات نصوص.
- **Model** (في `data`): يعرف شكل الخادم. `fromJson` يقرأ `lessions_count` المكتوبة خطأً في الخادم، ويحوّل `id` الرقمي إلى نص.

الوراثة (`CourseModel extends Course`) تجعل المستودع يعيد الـ Model مباشرة على أنه Entity بلا تحويل يدوي. الثمن ارتباط بسيط بين الصنفين. البديل الأنظف نظرياً: Model مستقل مع دالة `toEntity()` ، بثمن كود أكثر.

في مشروعي: أخطاء الخادم الإملائية (`lession` و `is_subscription`) محصورة في الـ Models. بقية التطبيق يرى `lessonsCount` و `isSubscribed` .

س66 لماذا Repository مجرد في الـ domain وتنفيذه في الـ data؟

متوسط

هذا **عكس الاعتماد** (Dependency Inversion): الـ domain يحدد العقد الذي يحتاجه، والـ data يلتزم به.

```
// domain
abstract class QuizRepository {
  Future<Either<Failure, QuizDetails>> getQuiz(String id);
}
// data
class QuizRepositoryImpl implements QuizRepository { ... }
```

النتيجة:

- الـ UseCase والـ Cubit لا يعرفان Dio ولا المسارات.
- في الاختبار أستبدل المستودع بمزيّف في سطر.
- لو أضفت تخزيناً محلياً (cache) يتغير التنفيذ فقط، لا من يستخدمه.

س67 ما فائدة الـ UseCase؟ أليس مجرد طبقة تمرير زائدة؟

متوسط

في هذا المشروع أغلب الـ UseCases فعلاً سطر واحد يستدعي المستودع، وهذا نقد مشروع. فائدتها:

- **اسم الفعل واضح:** `AttendExam` و `SubscribeWithCode` تقول ما يفعله التطبيق.
- **مكان طبيعي للمنطق** حين يكبر: دمج مستودعين، أو قاعدة عمل قبل الحفظ.
- **الـ Cubit يعتمد على ما يحتاجه فقط**، لا على مستودع بعشر دوال (Interface Segregation).

لو كان الوقت ضيقاً في مشروع صغير، لقبلت أن يستدعي الـ Cubit المستودع مباشرة. لكن الاتساق في 13 ميزة أهم عندي من توفير ملف.

س68 لماذا `Either<Failure, T>` بدل رمي الاستثناءات؟

متوسط

لأن الفشل يصبح **جزءاً من نوع الدالة**. لا يمكن للـ Cubit أن يأخذ النتيجة دون أن يقرر ماذا يفعل عند الفشل:

```
emit(result.fold(
  (failure) => state.copyWith(status: Status.failure, error: failure.message),
  (items)   => state.copyWith(status: Status.success, items: items),
));
```

مع الاستثناءات قد ينسى أحدهم `try/catch` ، فيبقى مؤشر التحميل إلى الأبد. `Left` هو الفشل، و `Right` النجاح. البديل الحديث `sealed class Result` في Dart 3 بلا حزمة خارجية.

س69 احكِ رحلة الخطأ من الخادم إلى الشاشة.

متوسط

```
DioException
  --ApiClient-->  ServerException / ValidationException / AuthException
  --guard------>  ServerFailure   / ValidationFailure   / AuthFailure
  --Cubit------>  state.error (نص عربي)  -->  ErrorView أو SnackBar
```

- `ApiClient._mapError` يحوّل رمز الحالة إلى استثناء برسالة عربية:
  - 422: أول خطأ في `errors`.
  - 401: «انتهت الجلسة».
  - 404: «المحتوى المطلوب غير موجود».
  - بلا رد: «تعذّر الاتصال».
- `guard` يحوّل الاستثناء إلى `Failure`.
- الـ Cubit يضع الرسالة في الحالة، والصفحة تعرض `ErrorView` بزر «إعادة المحاولة».

س70 لماذا لا يوجد `try/catch` في المستودعات ولا في الـ Cubits؟ وماذا عن الأخطاء البرمجية؟

متقدم

لأن `guard` يجمع المعالجة في مكان واحد. يلتقط `Exception` فقط (أخطاء متوقعة: شبكة، بيانات، تخزين)، و**لا يلتقط** `Error`، أي الأخطاء البرمجية مثل `RangeError` و`Null check`.

هذا مقصود: الخطأ البرمجي يجب أن يظهر ويُصلح أثناء التطوير، لا أن يتحول إلى رسالة «حدث خطأ» تخفيه.

> **في مشروعي:** استثناء: `ApiClient` يلتقط `TypeError` أثناء `parse` فقط، ويحوّله إلى «رد غير متوقع من الخادم». حقل غيّر الخادم نوعه ليس خطأً في كودي، ويجب ألا يجمّد الشاشة.

س71 كيف بدأت ببيانات تجريبية ثم ربطت الخادم دون تعديل الواجهة؟

متوسط

كل مصدر بيانات واجهة مجردة لها تنفيذان، وحقن الاعتماديات يختار بينهما. هذا نمط **Strategy**:

```
abstract class QuizRemoteDataSource { Future<QuizDetailsModel> getQuiz(String id); }
class QuizRemoteDataSourceImpl implements QuizRemoteDataSource { ... }  // الخادم
class QuizMockDataSource      implements QuizRemoteDataSource { ... }  // بيانات التصميم

// injection.dart: الربط = تغيير هذا السطر فقط
..registerLazySingleton<QuizRemoteDataSource>(() => QuizRemoteDataSourceImpl(sl()))
```

> **في مشروعي:** بعد ربط كل الوحدات حذفت المصادر التجريبية وبياناتها. بقي مصدر التنزيلات وحده، لأن حفظ الملفات على الجهاز لم يُبنَ بعد.

س72 ما حقن الاعتماديات (DI)؟ ولماذا `get_it`؟ وما الفرق بين أنواع التسجيل؟

متوسط

الأصناف لا تنشئ ما تعتمد عليه، بل تستقبله في المنشئ. فتُختبر بأي بديل، ويُجمع الربط في مكان واحد: `injection.dart`، بدالة `_initFeature()` لكل ميزة.

التسجيل	السلوك	أستخدمه لـ
`registerSingleton`	نسخة جاهزة فوراً	`SharedPreferences` المحمّلة عند البدء
`registerLazySingleton`	نسخة واحدة تُنشأ عند أول طلب	المصادر والمستودعات والـ UseCases (بلا حالة)
`registerFactory`	نسخة جديدة في كل طلب	الـ Cubits (حالة شاشة)

س73 `get_it` هو Service Locator، وكثيرون يعدّونه نمطاً سيئاً. ما ردك؟

متقدم

النقد صحيح حين يُستدعى `sl()` من أي مكان، فتصير الاعتماديات مخفية. لذلك قيّدته بقاعدة:

- يُستخدم في **نقطة التركيب** فقط: `injection.dart` والموجّه حين ينشئ Cubit.
- كل الأصناف تستقبل اعتمادياتها **في المنشئ** (constructor injection)، فتبقى صريحة وقابلة للاختبار.

الاستثناء الوحيد الموثّق: `redirect` في الموجّه يقرأ المستخدم المحفوظ، لأن `redirect` متزامن.

س74 ما قواعد الاعتماد بين الميزات المختلفة؟

متوسط

- **مسموح:** استخدام `domain` ميزة أخرى. الرئيسية تستخدم كيان `Course` من الدورات.
- **مسموح:** عناصر الواجهة المعدّة للمشاركة، مثل `ContentCard` .
- **ممنوع:** استيراد `data` ميزة أخرى. الـ Models والمصادر خاصة بميزتها.
- **ما تحتاجه ميزتان** ولا ينتمي لإحداهما ينتقل إلى `core/` ، مثل `Validators` و `HtmlText` .

> **في مشروعي:** الثمن: الرئيسية تقرأ بطاقة الدورة من ردها الخاص بدالة تكرر بعض قراءة `CourseModel` ، لأنها لا تستطيع استيراده. اخترت قليلاً من التكرار على ربط طبقتي بيانات ميزتين.

س75 لماذا رتبت المجلدات حسب الميزة (feature-first) لا حسب الطبقة؟

متوسط

لأن كل ما يخص «الاختبارات» في مجلد واحد: `features/quiz/{domain,data,presentation}` . حين أعمل على ميزة لا أتنقل بين ثلاثة مجلدات ضخمة. وحذف ميزة يعني حذف مجلدها. وفريق كبير يعمل بالتوازي على ميزات مختلفة دون تعارض. الترتيب حسب الطبقة يناسب مشروعاً صغيراً جداً، ويتضخم بسرعة.

س76 أعطني أمثلة على مبادئ SOLID من مشروعك.

متقدم

- **S**, مسؤولية واحدة: `ApiClient` للشبكة فقط، و `guard` للتحويل فقط، وكل UseCase فعل واحد.
- **O**, مفتوح للإضافة مغلق للتعديل: إضافة مصدر بيانات جديد لا تغيّر المستودع.
- **L**, الاستبدال: أي `QuizRepository` ، حقيقي أو مزيّف، يعمل مع الـ Cubit دون أن يعرف الفرق.
- **I**, فصل الواجهات: الـ Cubit يعتمد على UseCases محددة، لا على مستودع كامل.
- **D**, عكس الاعتماد: الـ domain يحدد واجهة المستودع، والـ data ينفّذها.

س77 ما أنماط التصميم (Design Patterns) التي استخدمتها؟

متقدم

النمط	أين
Repository	عزل مصدر البيانات خلف واجهة
Strategy	مصدر حقيقي أو تجريبي بنفس الواجهة
Adapter	`ApiClient` يكيّف Dio وغلاف الخادم لواجهة بسيطة، والـ Models تكيّف JSON
Factory	`fromJson`، وتسجيل الـ Cubits بـ `registerFactory`
Singleton	المستودعات والـ `ApiClient` عبر `get_it`
Observer	الـ Cubit يصدر حالات والواجهة تستمع، و`SessionExpiry` كـ `ChangeNotifier`
Command	كل UseCase أمر مستقل بـ `call()`

س78 أين تضع التحقق من المدخلات (Validation)؟

متوسط

- **في الواجهة:** تحقق فوري لتجربة الاستخدام، مثل زر معطّل أو رسالة تحت الحقل.
- **في المستودع:** تنظيف ثم تحقق قبل الطلب. لا يُرسل كود فارغ ولا تعليق من مسافات:

```
final trimmed = code.trim();
if (trimmed.isEmpty) return const Left(ValidationFailure(AppStrings.errorCodeRequired));
return guard(() => _remote.subscribeWithCode(trimmed));
```

- **في الخادم:** الحَكَم النهائي دائماً، ورسائل 422 منه تُعرض للمستخدم.

والقواعد المشتركة مثل رقم الهاتف في `core/utils/validators.dart`.

س79 كيف تقرأ ردود خادم لم تُوثَّق جيداً؟ صارم أم متسامح؟

متقدم

قاعدتي:

- **حقل رأيته في رد حقيقي يُقرأ بصرامة:** `json['title'] as String`. إن تغيّر نوعه يحوّله `ApiClient` إلى «رد غير متوقع» بدل بيانات خاطئة صامتة.
- **حقل لم أره يُقرأ بتسامح** مع قيمة افتراضية وتعليق يقول ذلك.

> **في مشروعي:** قائمة `/exams/my` لم تُرجع أي عنصر على خادم التدريب، فتُقرأ بغلاف `exam` أو بدونه، والحقول الغائبة بقيم افتراضية. والرد الحقيقي المحفوظ موضوع في اختبار البيانات بتاريخه.

الجزء 6

الشبكة والتعامل مع الـ API

كيف يتحدث التطبيق مع خادم Laravel: الترويسات، والتوكن، وغلاف الرد، والأخطاء، ورفع الملفات. سؤال «ماذا يحدث لو انقطع الإنترنت؟» شبه مؤكد.

س80 ما الطرق (methods) ورموز الحالة (status codes) التي تعاملت معها؟

أساسي

- `GET` للقراءة، و`POST` للإرسال. الخادم يستخدم POST حتى للتعديل، كما في ملف Postman.
- **200/201:** نجاح.
- **400:** خطأ منطق عمل برسالة، مثل «المستخدم ليس لديه فرع».
- **401:** غير مصادق، أي التوكن غائب أو منتهٍ.
- **404:** غير موجود.
- **422:** بيانات غير صالحة، مع `errors` لكل حقل.
- **500:** خطأ في الخادم.

س81 صِف `ApiClient` الذي كتبته. ما مسؤولياته؟

متوسط

العميل الوحيد للشبكة في التطبيق. كل طلب يمر منه، وله أربع مسؤوليات:

1. **الترويسات:** interceptor يضيف `Accept: application/json` ، و`Authorization: Bearer` إن وُجد توكن.
2. **فك الغلاف:** الخادم يرد `{message, data}` ، فيعيد `data` مباشرة.
3. **التحويل:** دالة `parse` تحوّل `data` إلى Models. وإن فشل التحويل لنوع غير متوقع يرمي «رد غير متوقع».
4. **الأخطاء:** يحوّل `DioException` إلى استثناءات طبقة البيانات برسائل عربية.

```
Future<List<CourseModel>> getCourses() => _client.get('/courses/paginate',
    parse: (data) => [for (final j in (data as List).cast<Map<String, dynamic>>())
                        CourseModel.fromJson(j)]);
```

س82 لماذا ترسل `Accept: application/json` في كل طلب؟

متوسط

لأن Laravel يقرر شكل رد الخطأ بحسب ما يطلبه العميل. بدون هذه الترويسة قد يعيد صفحة HTML لخطأ التحقق أو لإعادة التوجيه إلى صفحة الدخول، فيفشل فك JSON. معها يعيد دائماً JSON بالشكل الموثّق: 401 بـ `message` ، و422 بـ `errors` .

س83 أين يُحفظ التوكن؟ وكيف يصل لكل طلب؟

متوسط

- يُحفظ في `SharedPreferences` عبر `AuthLocalDataSource` بعد التحقق من الرمز.

- `ApiClient` لا يعرف مكان الحفظ. يأخذ دالة `readToken` تُقرأ **مع كل طلب**، فيصل التوكن الجديد فوراً بعد الدخول، ويختفي فوراً بعد الخروج.
- القراءة متزامنة لأن `SharedPreferences` تحمّل قيمها في الذاكرة عند بدء التطبيق.

تجنّب: في نسخة إنتاج أنقل التوكن إلى `flutter_secure_storage` (Keychain وKeystore). `SharedPreferences` نص غير مشفّر على الجهاز. أذكر هذا بنفسي قبل أن يُسأل.

س84 ماذا يحدث حين تنتهي الجلسة (401) أثناء استخدام التطبيق؟

متقدم

1. `ApiClient` يرى 401 على طلب **كان يحمل توكن**، فيستدعي `onUnauthorized`.
2. `SessionExpiry` (`ChangeNotifier`) يمسح الجلسة المحلية، ثم `notifyListeners()`.
3. الموجّه مربوط به `refreshListenable`، فيعيد تقييم `redirect`. لا مستخدم الآن، فيذهب إلى `/login`.

لماذا «كان يحمل توكن»؟ لأن 401 بلا توكن ليس انتهاء جلسة، فلا جلسة أصلاً، مثل طلب أثناء التسجيل. و`core` لا يعرف ميزة المصادقة: دالة المسح تُمرَّر له من حقن الاعتماديات.

س85 كيف تعرض رسائل أخطاء التحقق (422)؟

متوسط

رد Laravel لخطأ التحقق:

```
{ "message": "The name field is required. (and 2 more errors)",
  "errors": { "f_name": ["الحقل الاسم مطلوب"], "school_id": ["..."] } }
```

`message` يلحق به نص مثل «(and 2 more errors)». لذلك آخذ **أول رسالة في** `errors`، وهي عربية واضحة، وأعرضها. وإن لم توجد `errors` أرجع إلى `message`.

في مشروعي: حيلة مفيدة تعلمتها: إرسال طلب فارغ عمداً يكشف كل قواعد التحقق في رد 422 دون أن يحفظ الخادم شيئاً.

س86 ماذا يحدث لو انقطع الإنترنت؟

أساسي

- Dio يفشل بلا رد، أو يتجاوز المهلة: 15 ثانية للاتصال و20 للاستقبال.
- `ApiClient` يحوّله إلى «تعذّر الاتصال بالخادم، تحقق من اتصالك بالإنترنت».
- الشاشة تعرض `ErrorView` بالرسالة وزر «إعادة المحاولة» الذي يستدعي `load()` من جديد.
- الصور المحفوظة سابقاً تظهر من ذاكرة `cached_network_image`.

في مشروعي: كل شاشة لها حالاتها الأربع: تحميل، فارغ، خطأ مع إعادة المحاولة، نجاح. وهذا بند في قائمة الانتهاء لكل ميزة.

س87 لماذا ترسل البيانات `form-data` وليس JSON؟ وكيف ترسل مصفوفة؟

متوسط

لأن الخادم يقرأ الحقول هكذا، كما في Postman. وأي شكل آخر يُرفض بـ 422. المصفوفة تُرسل بتكرار المفتاح مع `[]`:

```
_client.post('/exams/attend-exam', fields: {'answers_id[]': answerIds});
// يصبح: answers_id[]=202&answers_id[]=206
```

س88 كيف رفعت صورة الملف الشخصي؟ متوسط

طلب `multipart/form-data` بالحقل `image` . أرفع **البايتات** لا مسار الملف، لأن المسار لا يوجد على الويب.

والبايتات تعمل على كل المنصات.

```
FormData.fromMap({'image': MultipartFile.fromBytes(bytes,
    filename: fileName, contentType: DioMediaType.parse('image/jpeg'))});
```

والصورة تُختار بـ `image_picker` ، والخادم يعيد المستخدم كاملاً بعد الرفع، فأحدّث الجلسة المحلية به.

س89 الخادم لا يعيد صفحات (pagination). كيف كنت ستطبق التمرير اللانهائي لو أضافه؟ متقدم

- الحالة تحمل: `items` و `page` و `hasMore` و `loadingMore` .
- `ScrollController` أو عنصر أخير في `ListView.builder` يطلب الصفحة التالية قبل النهاية بمسافة.
- الصفحة الجديدة تُضاف للقائمة، وفشلها لا يمسح ما سبق: حالة تحميل الصفحات منفصلة عن الحالة الأولى.
- أمنع الطلب المكرر بفحص `loadingMore` .

> **في مشروعي:** اكتشفت أن `*/paginate` في هذا الخادم يعيد مصفوفة مجردة لا paginator رغم اسمه، فقرأته كقائمة كاملة ووثّقت ذلك.

س90 كيف تضيف عملاً بلا إنترنت (offline) أو تخزيناً مؤقتاً للبيانات؟ متقدم

في طبقة البيانات فقط، والواجهة لا تتغير:

- **مصدر محلي** (`LocalDataSource` بـ Hive أو sqflite أو drift) بجانب المصدر البعيد.
- **المستودع يقرر**، والنمط الشائع stale-while-revalidate: يعرض المحفوظ فوراً، ثم يطلب الجديد ويحدّث. يمكن أن يعيد `Stream` بقيمتين.
- الأفعال (حضور اختبار) تُحفظ في طابور، وتُرسل عند عودة الاتصال.

هذا بالضبط ما تسهّله Clean Architecture: التغيير محصور في `data` .

س91 لماذا كتبت `fromJson` يدوياً بدل `json_serializable` أو `freezed` ؟ متوسط

- **اليدوي:** بلا توليد كود ولا `build_runner` ، وتحكم كامل في الحالات الشاذة. مثل مدة الدرس نصاً حراً، و `current_answer` رقماً (0 أو 1) أو منطقياً، ومعرّف رقمي يصير نصاً.
- **التوليد:** كود أقل وأخطاء كتابة أقل في المشاريع الكبيرة. و `freezed` يولّد `copyWith` والمساواة والـ sealed unions أيضاً.

لعدد Models متوسط وخادم غير منتظم كان اليدوي أوضح. في مشروع أكبر سأستخدم `freezed` للحالات والكيانات.

س92 كيف تصحح مشكلة في طلب شبكة؟

متقدم

1. أعيد الطلب في **Postman** بالتوكن نفسه، لأعرف هل المشكلة في الخادم أم في التطبيق.
2. أراقب الطلبات في **تبويب Network في Flutter DevTools**، أو بـ `LogInterceptor` من Dio يُضاف في وضع debug فقط.
3. أكتب اختبار بيانات بالرد الحقيقي، فيتحول الخطأ إلى اختبار يمنع رجوعه.

> **تجنّب:** لا تترك طباعة التوكن أو الردود في نسخة release. مشروعي خالٍ من `print` ، وهذا يُفحص في المراجعة.

الجزء 7

التنقل والجلسة: go_router

كيف ينتقل المستخدم بين الشاشات، وكيف تُحمى الصفحات بلا جلسة، وكيف تحفظ التبويبات السفلية حالتها.

س93 لماذا go_router بدل `Navigator.push` المباشر؟

أساسي

- **مسارات نصية** مثل `/courses/15`، فتعمل الروابط العميقة (deep links) والويب تلقائياً.
- **حارس مركزي** (`redirect`) بدل فحص الجلسة في كل شاشة.
- **تبويبات سفلية** تحفظ حالة كل تبويب (`StatefulShellRoute`).
- **كل المسارات في ملف واحد**، وهو المكان الذي يُنشأ فيه الـ Cubit لكل صفحة.

س94 ما الفرق بين `push` و`go` و`pushReplacement` و`pop`؟

متوسط

- `context.push('/x')`: يضيف صفحة فوق المكدّس، والرجوع يعود للسابقة.
- `context.go('/x')`: ينتقل إلى المسار ويبني المكدّس من جديد بحسب شجرة المسارات، فتضيع الصفحات السابقة.
- `context.pushReplacement('/x')`: يستبدل الصفحة الحالية بأخرى.
- `context.pop()`: يعود خطوة، ويمكن أن يعيد قيمة لمن فتح الصفحة.

> **في مشروعي:**
> - بعد الدخول: `go('/home')`، حتى لا يعود زر الرجوع لصفحة الرمز.
> - بعد إرسال الاختبار: `go` إلى النتيجة، حتى لا يعود الطالب لأسئلة أجاب عنها.
> - الانتقال من درس لدرس آخر في صفحة الدرس: `pushReplacement`، حتى لا يتكدس عشرة دروس فوق بعضها.

س95 لماذا تمرّر المعرّفات في المسار لا في `extra`؟

متوسط

`extra` كائن في الذاكرة لا يظهر في الرابط. يضيع إن:

- فُتحت الصفحة من رابط عميق أو إشعار.
- أُعيد بناء التطبيق على الويب.
- استعاد النظام التطبيق بعد قتله.

لذلك `/quiz/6/result` يحمل المعرّف، وصفحة النتيجة تجلب بياناتها من الخادم بنفسها.

> **في مشروعي:** كانت صفحة النتيجة تأخذ الدرجة عبر `extra` فغيّرتها. بقي استثناء واحد أعرفه: رقم الهاتف بين الدخول والرمز والتسجيل يمر بـ `extra`، ولو فُتحت صفحة الرمز مباشرة لن تجده. تحسينه: وضعه في حالة الجلسة أو في query.

س96 كيف يمنع الموجّه دخول الصفحات بلا جلسة؟

متوسط

```
redirect: (context, state) {
  final location = state.matchedLocation;
  if (location == '/') return null;                    // شاشة البداية تقرر بنفسها
  final loggedIn = sl<AuthRepository>().cachedUser != null;
  final onAuthRoute = {'/login', '/register', '/otp'}.contains(location);
  if (!loggedIn && !onAuthRoute) return '/login';
  if (loggedIn && onAuthRoute) return '/home';
  return null;                                         // اسمح
},
refreshListenable: sl<SessionExpiry>(),                // يعيد التقييم عند انتهاء الجلسة
```

`redirect` يُستدعى قبل كل انتقال. وهو متزامن، لذلك يقرأ المستخدم المحفوظ في الذاكرة لا من الخادم.

س97 كيف تحفظ التبويبات السفلية حالتها؟

متوسط

`StatefulShellRoute.indexedStack` : لكل تبويب (الرئيسية، المعرض، موادي، الملف) فرع بمكدّس تنقل مستقل. وكلها حية داخل `IndexedStack` ، فالرجوع لتبويب يجده كما تُرك: موضع التمرير، والبيانات، والصفحات المفتوحة داخله. و `AppShell` يرسم الشريط السفلي ويبدّل الفرع بـ `navigationShell.goBranch(i)` .

س98 لماذا يُنشأ الـ `GoRouter` في منشئ `DarbApp` لا داخل `build` ؟

متقدم

لأن `build` قد يُستدعى مرات كثيرة، مثل تغيير اتجاه الشاشة أو السمة. إنشاء موجّه جديد في كل مرة يعيد التنقل إلى البداية ويضيّع المكدّس. القاعدة: الكائنات ذات العمر الطويل (موجّه، متحكمات، مؤقتات) لا تُنشأ في `build` .

س99 ماذا تفعل شاشة البداية (Splash)؟

متوسط

تعرض الشعار، ثم تقرر الوجهة: إن وُجد مستخدم محفوظ تذهب إلى `/home` ، وإلا إلى `/login` . لذلك يستثنيها `redirect` (`location == '/'`)، لتعرض الشعار أولاً بدل أن يحوّلها فوراً.

س100 كيف تفعّل الروابط العميقة (Deep Links)؟

متقدم

- **Android:** `intent-filter` في `AndroidManifest.xml` للنطاق، مع ملف `assetlinks.json` على الخادم لروابط App Links.
- **iOS:** Associated Domains مع ملف `apple-app-site-association` .
- go_router يستقبل المسار ويطابقه مع المسارات الموجودة، ويطبّق `redirect` عليه أيضاً.

لهذا كان مهماً أن تحمل المسارات معرّفاتها، فيعمل `/courses/15` من رابط مباشرة.

الجزء 8

الواجهة ومطابقة تصميم فيجما

المطلوب في الدورة تطبيق يطابق التصميم. هنا تشرح كيف حوّلت إطارات فيجما إلى ويدجتات دقيقة، وكيف نظّمت الألوان والنصوص والأيقونات والعناصر المشتركة.

س101 كيف تأكدت أن الشاشات تطابق تصميم فيجما فعلاً؟

متوسط

1. حفظت إطارات فيجما صوراً بعرض الهاتف في التصميم (430) في `docs/design/figma` ، مع فهرس يربط كل إطار بمساره في التطبيق.
2. قست المسافات والمقاسات والألوان من الصور مباشرة.
3. رسمت الشاشة في اختبار golden بالعرض نفسه وبيانات التصميم نفسها، ثم وضعتها بجانب إطار فيجما في صورة واحدة.
4. عدّلت حتى تتطابق المواضع، وكررت المقارنة.

هكذا وجدت فروقاً لا تراها العين في المحاكي: أيقونة في الطرف الخطأ، وارتفاع ترويسة يزيد 20 بكسل، وزوايا 20 بدل 10.

س102 لماذا جمعت الألوان والنصوص والأيقونات في ملفات مركزية؟

متوسط

- `AppColors` : كل لون باسم يصف مكانه (`answerFill` و `reviewWrongBar`). تغيير لون في التصميم سطر واحد.
- `AppStrings` : كل نص عربي في مكان واحد. المراجعة اللغوية وإضافة لغة ثانية لاحقاً أسهل، ولا نصوص مبعثرة في الويدجتات.
- `app_icons.dart` : أيقونات Phosphor بأسمائها كما في التصميم.

القاعدة في المنهج: لا `Color(0x...)` ولا نص عربي داخل الواجهة. وفي التنظيف حذفت 24 نصاً و5 ألوان لم يعد يستخدمها شيء، ودمجت النصوص المكررة.

س103 كيف أضفت أيقونات التصميم؟ ولماذا خط أيقونات لا صور SVG؟

متوسط

التصميم يستخدم مجموعة **Phosphor**. ضمّنت خطيها (Regular وFill) في المشروع، وعرّفت كل أيقونة برمزها:

```
static const coins = IconData(0xe78e, fontFamily: 'Phosphor');
Icon(PhosphorIconsRegular.coins, color: AppColors.iconBlue, size: 16)
```

- الخط يتلوّن ويتحجّم كأي نص.
- في نسخة release يُقلّص تلقائياً إلى الأيقونات المستخدمة فقط: من 488KB إلى 15KB.
- حزمة `phosphor_flutter` نفسها لم تتوافق مع إصدار Flutter الحالي، فضمّنت الخط مباشرة، وتحققت أن كل رمز موجود فعلاً في ملف الخط.

الرسوم الملوّنة المركّبة، مثل أيقونة الاختبار والشعار، بقيت **SVG** عبر `flutter_svg` .

س104 ما العناصر المشتركة التي بنيتها؟ ولماذا؟

متوسط

- `PageHeader` : الترويسة الفيروزية، بالشعار أو سهم الرجوع، والعنوان، وزر الإشعارات، وحقل أسفلها.
- `ErrorView` و `EmptyView` و `LoadingView` : الحالات الموحّدة في كل الشاشات.
- `ContentCard` : بطاقة الدورة والاختبار والنشاط، في صفحة المادة والبحث و«اختباراتي».
- `showConfirmDialog` و `showAppSheet` : الحوارات والأوراق السفلية.

السبب أن التصميم يكرر هذه الأشكال. لو بُنيت في كل شاشة على حدة لاختلفت تفاصيلها مع الوقت. وبالتركيب (composition) يأخذ العنصر ما يختلف معاملاً، مثل `radius` و `borderColor` ، ويبقى الباقي موحّداً.

س105 التصميم يُظهر الشاشة وهي مليئة بالبيانات فقط. ماذا عن الحالات الأخرى؟

أساسي

لكل شاشة أربع حالات، وأبنيها كلها:

- **تحميل:** `LoadingView` .
- **فارغ:** `EmptyView` برسالة مناسبة، مثل «لا توجد اختبارات بعد».
- **خطأ:** `ErrorView` برسالة الخادم وزر إعادة المحاولة.
- **نجاح:** كما في التصميم.

وحين لا يغطي التصميم حالة، أتبع أقرب نمط موجود في التطبيق، لتبقى التجربة متسقة.

```
builder: (context, state) => switch (state.status) {
  Status.failure => ErrorView(message: state.error, onRetry: cubit.load),
  Status.success when state.items.isEmpty => const EmptyView(message: ...),
  Status.success => _List(items: state.items),
  _ => const LoadingView(),
},
```

س106 ماذا فعلت لإمكانية الوصول (Accessibility)؟

متقدم

- `Semantics(selected: ...)` على التبويبات والخيارات، فيعلن قارئ الشاشة أيها المختار.
- `Tooltip` على أزرار الأيقونات وحدها (رجوع، إشعارات، مشاركة، تشغيل)، فلها اسم مسموع.
- لا ارتفاعات ثابتة للنصوص، وحدود دنيا بدل ثابتة للبطاقات، فيتحمل التصميم تكبير الخط.
- مساحات ضغط لا تقل عن نحو 40 بكسل.

> **تجنّب:** التباين في بعض نصوص التصميم الرمادية الفاتحة ضعيف. أعرف هذا، وألتزم بالتصميم لأنه المطلوب، مع ذكره كتحسين.

س107 كيف تعرض الصور القادمة من الخادم؟

متوسط

بـ `CachedNetworkImage` : تُحفظ على الجهاز بعد أول تحميل، ولها:

- `placeholder` بلون هادئ أثناء التحميل.

- `errorWidget` بأيقونة إن فشل الرابط.

هكذا لا يظهر مربع فارغ ولا خطأ أحمر للمستخدم. وصور المواد دائرية بعنصر مشترك `CircleImage` .

س108 لماذا خط Noto Sans Arabic مع Lato؟

متوسط

هما خطا التصميم في فيجما. Noto للعربية، وLato احتياطي (`fontFamilyFallback`) للحروف اللاتينية والأرقام، كما تظهر «85%» و«22:30». ضمّنت الخطين في المشروع بأوزانها، لأن خطوط النظام تختلف بين الأجهزة.

س109 كيف تضيف الوضع الداكن أو لغة ثانية؟

متقدم

- **الداكن:** `AppTheme.dark()` و `themeMode` . الألوان المكتوبة مباشرة في `AppColors` تحتاج أن تصبح «رموزاً» دلالية تُقرأ من السمة (`ThemeExtension`)، بدل قيم ثابتة.
- **اللغة:** نقل `AppStrings` إلى ملفات ARB مع `flutter gen-l10n` ، فتصير `context.l10n.title` . التخطيط جاهز أصلاً لأنه بـ start وend.

> **في مشروعي:** التصميم عربي فاتح فقط، فلم أبنهما. لكن جمع الألوان والنصوص في مكان واحد يجعل إضافتهما عملاً محصوراً لا إعادة كتابة.

س110 هل استخدمت حركات (Animations)؟

متوسط

- **ضمنية** (implicit) مثل `AnimatedContainer` : تبديل لون التبويب المختار بانتقال ناعم. أعطيها القيمة الجديدة والمدة فقط.
- **صريحة** (explicit) بـ `AnimationController` : حين أحتاج تحكماً كاملاً، كالتكرار والتسلسل والإيقاف. وتحتاج `dispose` للمتحكم.

التصميم لم يطلب حركات معقدة، فاكتفيت بالضمنية حيث يتغير شكل عنصر.

الجزء 9

الاختبارات

وجود 338 اختباراً نقطة قوة كبيرة، لكن المناقش سيسأل: ماذا تختبر؟ وكيف تختبر بلا خادم؟ ولماذا هذا الأسلوب؟

س111 ما أنواع الاختبارات في Flutter؟

أساسي

النوع	يختبر	السرعة
Unit	دالة أو صنف بلا واجهة: Model، مستودع، Cubit، دالة مثل `learningPath`	الأسرع
Widget	شاشة أو عنصر في بيئة اختبار بلا جهاز: يضغط ويكتب ويفحص ما يظهر	سريع
Integration	التطبيق كاملاً على جهاز أو محاكٍ	بطيء
Golden	صورة الواجهة مقارنة بصورة مرجعية	سريع، حساس للخطوط والمنصة

س112 ماذا اختبرت في مشروعك؟

متوسط

338 اختباراً. لكل ميزة ثلاثة ملفات:

- `<feature>_data_test.dart` : الـ Model على رد حقيقي محفوظ، والمسار والحقول المرسلة، وردود الخطأ 422 و400 و404.
- `<feature>_cubits_test.dart` : تسلسل الحالات في النجاح والفشل، بمستودع مزيّف.
- `<feature>_flow_test.dart` : الشاشة كما يستخدمها المستخدم: يفتح، ويضغط، ويرى النتيجة.

إضافة إلى اختبار يفتح كل مسار في التطبيق، واختبارات وحدة لدوال مثل `learningPath` وتحويل HTML.

س113 كيف تختبر طبقة البيانات بلا خادم؟

متوسط

بمحوّل HTTP مزيّف (`FakeHttpAdapter`) يُركَّب في Dio بدل الشبكة. يعيد ما أحدده لكل مسار، ويسجّل كل طلب:

```
adapter.on('/subscriptions/my', body: {'message': 'ok', 'data': {'subjects': [...]}});
final courses = (await repository.getMyCourses()).getOrElse(() => []);
expect(courses.map((c) => c.id), ['1']);
expect(adapter.requests.single.queryParameters, isEmpty);
```

والـ JSON في الاختبار هو **رد حقيقي** نسخته من الخادم مع تاريخه. فإن غيّر الخادم شكله لاحقاً، أعرف بالضبط ما كان عليه.

س114 كيف تختبر Cubit؟

متوسط

بمستودع مزيّف (`Fake`) يعيد ما أريد، مع UseCases **حقيقية**، ثم أفحص الحالة النهائية أو تسلسل الحالات:

```
test('الفشل يصل برسالته', () async {
  final cubit = NotificationsCubit(GetNotifications(_FakeRepo(const
Left(ServerFailure('x')))));
  await cubit.load();
  expect(cubit.state, const NotificationsState(status: NotificationsStatus.failure, error:
'x'));
});
```

`Equatable` يجعل مقارنة الحالة كاملة بسطر واحد ممكنة.

س115 لماذا Fakes مكتوبة يدوياً بدل Mocks (mockito أو mocktail)؟

متوسط

- **Fake:** تنفيذ بسيط حقيقي للواجهة، يعيد قيماً ويسجّل ما وصله. يُقرأ كأنه كود عادي، ويختبر **النتيجة**.
- **Mock:** يُبرمج بـ `when(...).thenReturn(...)` ، ويفحص **كيف** استُدعي (`verify`). قوي، لكنه يربط الاختبار بتفاصيل التنفيذ فيتكسر مع إعادة الهيكلة.

واجهاتي صغيرة (مستودع بدوال قليلة)، فالـ Fake أوضح ولا يحتاج حزمة. أستخدم mocktail حين تكون الواجهة كبيرة أو أريد التحقق من عدد الاستدعاءات.

س116 اشرح اختبار واجهة (widget test) كتبته.

متوسط

```
testWidgets('النتيجة معطّلة حتى تُجاب كل الأسئلة', (tester) async {
  repository.quizLoads.addAll([Right(_taking), Right(_scored)]);
  await pump(tester, '/quiz/6');
  expect(tester.widget<ElevatedButton>(find.byType(ElevatedButton)).onPressed, isNull);

  await tester.tap(find.text('110'));  await tester.pump();
  await tester.tap(find.text('333'));  await tester.pump();
  expect(tester.widget<ElevatedButton>(find.byType(ElevatedButton)).onPressed, isNotNull);

  await tester.tap(find.text('النتيجة'));  await tester.pumpAndSettle();
  expect(repository.attended, ['202', '206']);   // أُرسلت الإجابات الصحيحة
  expect(find.text('50%'), findsOneWidget);       // وصلت صفحة النتيجة
});
```

- `pump()` : إطار واحد.
- `pumpAndSettle()` : حتى تنتهي الحركات والانتقالات.
- `find` يبحث في الشجرة، و `expect` يتحقق.

س117 كيف تختبر شيئاً يعتمد على الوقت، مثل تأخير البحث 400ms؟

متقدم

- جعلت مدة التأخير **معاملاً** في الـ Cubit (`debounce`)، فأمرّر مدة قصيرة في الاختبار.
- في اختبارات الواجهة `tester.pump(const Duration(milliseconds: 400))` يقدّم الساعة الوهمية دون انتظار حقيقي.

- البديل `fakeAsync` من حزمة `fake_async` للتحكم الكامل بالزمن في اختبارات الوحدة.

> **في مشروعي:** لاختبار «الرد القديم لا يُعرض» استخدمت `Completer` في المستودع المزيّف: أؤخّر رد البحث الأول، وأكمل الثاني قبله، ثم أتحقق أن الحالة تعرض نتيجة الثاني.

س118 هل استخدمت اختبارات Golden؟

متقدم

استخدمتها **أداة مؤقتة** لمطابقة التصميم: أرسم الشاشة بعرض 430 وبيانات التصميم، وأضعها بجانب إطار فيجما. لم أُبقها في المستودع، لأن صور golden حساسة للخطوط ونظام التشغيل، فقد تفشل على جهاز آخر أو في CI دون خطأ حقيقي. لإبقائها دائماً تُثبَّت الخطوط في الاختبار، وتُشغَّل على منصة واحدة ثابتة (مثل حاوية Linux في CI).

> **تجنّب:** في بيئة الاختبار الظلال لا تُرسم بضبابية، والخط غير المحمّل يظهر مربعات. صورة golden ليست دائماً ما يراه المستخدم.

س119 ما اختبار المسارات (routes smoke test)؟

متوسط

اختبار يفتح كل مسار في الموجّه الحقيقي بمستودعات مزيّفة، ويتحقق أن الصفحة تُبنى بلا استثناء وتخرج من حالة التحميل. يلتقط أخطاء التركيب: Cubit غير مسجّل، أو مسار بلا `BlocProvider` ، أو معامل مسار ناقص. وهي أخطاء لا تكشفها اختبارات الصفحة منفردة.

س120 هل كتبت اختباراً قبل إصلاح خطأ؟

متوسط

نعم، في خطأ الإجابات الصحيحة المتعددة:

1. كتبت اختباراً بسؤال له إجابتان صحيحتان واختيار الثانية.
2. رأيته يعرض «خاطئة».
3. أصلحت الشرط.
4. رأيته ينجح.

هذا جوهر TDD للأخطاء: الاختبار يثبت وجود الخطأ أولاً، ثم يمنع رجوعه للأبد.

س121 ما الذي لا تغطيه اختباراتك؟

متقدم

- **لا اختبارات تكامل على جهاز حقيقي** (`integration_test`). الإضافات الأصلية، مثل مشغّل يوتيوب واختيار الصور، مستبدلة في اختبارات الواجهة.
- **الخادم الحقيقي لا يُستدعى في الاختبارات** عمداً، حتى تبقى سريعة وثابتة. جرّبته يدوياً، وأحفظ ردوده في اختبارات البيانات.
- **لا قياس للتغطية (coverage)** كرقم. أضيفه بـ `flutter test --coverage` .

> **تجنّب:** لا تدّعِ تغطية 100%. الصراحة في الحدود تُحسب لك.

س122 كيف تشغّل الفحوص تلقائياً مع كل Pull Request؟

متقدم

بـ GitHub Actions: ملف `.github/workflows/ci.yml` يثبّت Flutter، ثم `flutter pub get`، ثم `flutter analyze`، ثم `flutter test`. ويُجعل نجاحه شرطاً للدمج في إعدادات حماية الفرع.

> **في مشروعي:** القاعدة الآن يدوية: `flutter analyze` و `flutter test` نظيفان قبل أي commit، وهي مكتوبة في تعليمات المشروع. نقلها إلى CI خطوتي التالية.

الجزء 10

الأداء

لا يكفي أن يعمل التطبيق، يجب أن يعمل بسلاسة. أسئلة الأداء تكشف إن كنت تعرف ما يحدث خلف الشاشة.

س123 كيف تتجنب إعادة البناء غير الضرورية؟

متوسط

- `const` للويدجتات الثابتة: يتخطاها Flutter.
- **تقسيم الويدجتات** إلى أصناف صغيرة بدل دوال تعيد ويدجتات، فيُعاد بناء الجزء المتغير فقط.
- `context.select` و `buildWhen` : الاشتراك فيما يهم الويدجت من الحالة فقط.
- `Equatable` في الحالات: الحالة المساوية لا تعيد البناء.
- `setState` في أصغر ويدجت تحتاجه.

س124 لا تضع عملاً ثقيلاً في `build` . أين تضعه إذن؟

متوسط

`build` قد يُستدعى 60 مرة في الثانية أثناء الحركة. لذلك يُجهَّز كل حساب قبل الواجهة:

- تحويل HTML نص السؤال يتم **مرة واحدة في** `fromJson` ، لا عند كل رسم.
- `learningPath` تحسب حالات الدروس من البيانات، والواجهة ترسم النتيجة.
- لا طلبات شبكة ولا قراءة ملفات في `build` . هذه في الـ Cubit عند الفتح.

س125 كيف تعرض صوراً كثيرة دون استهلاك الذاكرة؟

متوسط

- `CachedNetworkImage` : تخزين على القرص، فلا تحميل متكرر من الشبكة.
- `memCacheWidth` أو `cacheWidth` : فك الصورة بحجم عرضها لا بحجمها الأصلي. صورة 4000 بكسل في دائرة 110 بكسل تستهلك ذاكرة هائلة بلا داعٍ.
- القوائم الكسولة (`ListView.builder` و `GridView.builder`) لا تفك إلا الصور الظاهرة.

> **تجنّب:** صور المعرض من الخادم بأحجام كاملة. إضافة `memCacheWidth` للمصغّرات تحسين أعرفه ولم أطبقه بعد.

س126 كيف تكتشف مشكلة أداء؟ ما أدواتك؟

متقدم

1. أشغّل في **وضع profile** على جهاز حقيقي، لأن debug أبطأ بطبيعته ولا يُقاس عليه.
2. **Flutter DevTools**:
   - Performance: الإطارات البطيئة، وأيها بسبب build أو raster.
   - CPU Profiler: الدالة الثقيلة.
   - Memory: التسريبات.

- Rebuild Stats: الويدجتات التي يُعاد بناؤها كثيراً.

3. **Performance Overlay** يرسم زمن كل إطار فوق الشاشة.

القاعدة: **أقيس أولاً ثم أحسّن**، ولا أحسّن بالتخمين.

س127 ما مصادر تسريب الذاكرة في Flutter؟ وكيف تمنعها؟

متوسط

- متحكمات لا تُغلق: `TextEditingController` و `AnimationController` و `ScrollController`. تُغلق في `dispose`.
- مؤقتات واشتراكات `Stream` تعمل بعد إغلاق الصفحة.
- Cubits لا تُغلق. `BlocProvider` يغلقها تلقائياً.
- دوال callback تحتفظ بـ `context` صفحة مغلقة.

> **في مشروعي:** `SearchCubit` يلغي مؤقت التأخير في `close()`، وشاشة البداية تلغي مؤقتها في `dispose`، ومتحكم حقل التعليق يُغلق مع صفحة الدرس.

س128 حجم نسخة APK عندك 58MB. كيف تقلّصه؟

متقدم

- الـ APK الحالي «سمين» (fat): يحوي الكود المترجم لثلاث معماريات معالج.
- `flutter build apk --split-per-abi` ينتج ملفاً لكل معمارية، نحو ثلث الحجم.
- `flutter build appbundle` للمتجر: Google Play يرسل لكل جهاز ما يحتاجه فقط.
- حذف الأصول غير المستخدمة. في التنظيف حذفت 8 صور (نحو 500KB) لم يعد يستخدمها شيء.
- تقليص خطوط الأيقونات يحدث تلقائياً (من 488KB إلى 15KB).
- `--obfuscate --split-debug-info` يصغّر الكود ويصعّب قراءته.

س129 كيف تجعل التطبيق يبدأ بسرعة؟

متقدم

- `main` ينتظر فقط ما لا بد منه: `SharedPreferences`، لأن الموجّه يقرأ الجلسة منها بشكل متزامن.
- كل شيء آخر `registerLazySingleton`: لا يُنشأ إلا عند أول استخدام.
- البيانات تُطلب عند فتح كل صفحة، لا كلها عند البدء.
- شاشة البداية تغطي هذه اللحظات بالشعار.

الجزء 11

الأمان

السينيور يسأل عن الأمان ليعرف إن كنت تفهم أين تُحمى البيانات فعلاً. الجواب الذهبي: التطبيق ليس حدّ الأمان، الخادم هو الحد.

س130 قفلت الدروس المدفوعة في التطبيق. هل هذا آمن؟

متوسط

لا. القفل في التطبيق تجربة استخدام فقط، لا حماية. أي شخص يستطيع:

- قراءة ردود الشبكة بأداة وسيطة.
- تعديل التطبيق، أو استدعاء الـ API مباشرة.

اكتشفت أن رد الدرس المقفل يحتوي رابط الفيديو، وأن رد الاختبار يحتوي الإجابات الصحيحة قبل الحل. الحل الصحيح في **الخادم**: ألا يرسل الرابط إلا لمشترك، وألا يرسل الإجابات إلا بعد الحضور. وثّقت المشكلتين لمطوّر الواجهة الخلفية.

> **تجنّب:** لا تقل «أخفيت الرابط في الواجهة فصار آمناً». هذا أول ما يكشفه المناقش.

س131 أين تحفظ التوكن؟ وهل هذا آمن؟

متوسط

حالياً في `SharedPreferences` ، وهو ملف غير مشفّر داخل مساحة التطبيق. محمي من التطبيقات الأخرى في الجهاز العادي، لكنه مكشوف على جهاز مكسور الحماية (root أو jailbreak) أو في نسخة احتياطية.

للإنتاج أنقله إلى `flutter_secure_storage` : يستخدم Keychain في iOS، وKeystore في Android. مع بقاء قراءة متزامنة عبر نسخة في الذاكرة تُحمّل عند البدء، لأن `ApiClient` يقرأ التوكن مع كل طلب.

س132 هل يمكن وضع مفاتيح سرية (API keys) داخل التطبيق؟

متقدم

لا إن كانت سرية فعلاً. أي شيء داخل التطبيق يمكن استخراجه، حتى مع `--obfuscate` ، لأن التشويش يصعّب القراءة ولا يمنعها. الأسرار تبقى في الخادم، والتطبيق يحصل على توكن مؤقت خاص بالمستخدم.

> **في مشروعي:** التطبيق لا يحوي أي سر. عنوان الخادم فقط، ويُغيَّر وقت البناء بـ `--dart-define=API_BASE_URL=...` دون تعديل الكود.

س133 ماذا يحدث عند تسجيل الخروج؟

متوسط

1. `GET auth/logout` يُبطل التوكن في الخادم، فلا يعمل حتى لو سُرق بعد الخروج.
2. يُمسح التوكن والمستخدم من الجهاز.
3. الموجّه يرى أن لا مستخدم، فيحوّل إلى صفحة الدخول.

معرّف الجهاز يبقى، لأن الخادم يربطه بالجهاز لا بالجلسة.

س134 رمز التحقق في خادم التدريب ثابت (1111). هل هذا في التطبيق؟

متوسط

لا. هذا سلوك خادم التدريب فقط لتسهيل التجربة، وليس في كود التطبيق أي رمز أو رقم هاتف مكتوب. في الإنتاج يرسل الخادم رمزاً حقيقياً، ويجب أن يحدّ من المحاولات (rate limiting) حتى لا يُخمَّن الرمز.

س135 نص السؤال يصل HTML من الخادم. هل هناك خطر؟

متقدم

لو عرضته في WebView كما هو، لأمكن حقن سكربت (XSS) إن كتب أحدهم محتوى خبيثاً في لوحة التحكم. أنا أحوّله إلى **نص عادي**: أزيل الوسوم، وأفك الرموز مثل ` `، وأحوّل الفقرات إلى أسطر. ثم أعرضه في `Text`، الذي لا ينفّذ شيئاً. الدالة في `core/utils/html_text.dart` ولها اختبارات.

س136 ما الصلاحيات التي يطلبها التطبيق؟

متوسط

- **Android:** `INTERNET` فقط. اختيار الصور يتم بمنتقي النظام دون صلاحية تخزين كاملة.
- **iOS:** `image_picker` يحتاج نص `NSPhotoLibraryUsageDescription` في `Info.plist`، وهو موجود، وإلا ينهار التطبيق عند الطلب أو يرفضه المتجر. ولو أضيف التصوير بالكاميرا يلزم `NSCameraUsageDescription` أيضاً.

مبدأ أقل صلاحية: لا أطلب إلا ما تحتاجه ميزة موجودة.

الجزء 12

طريقة العمل وGit والنشر

كيف تعمل كمهندس في فريق: الفروع، والـ commits، والمراجعة، والتجهيز للنشر. هذه الأسئلة تُظهر إن كنت جاهزاً لبيئة عمل حقيقية.

س137 كيف نظّمت العمل في Git؟

أساسي

- لا عمل مباشر على `main`. لكل ميزة أو إصلاح فرع: `feat/quiz-backend` أو `fix/lesson-video-hang`.
- commit صغير بعد كل خطوة، ثم Pull Request يُراجع قبل الدمج.
- إن لم يُدمج الـ PR السابق بعد، أفتح الفرع الجديد منه، حتى لا أنتظر.

س138 ما صيغة رسائل الـ commit التي استخدمتها؟

متوسط

صيغة **Conventional Commits**: `type(scope): description` بالإنجليزية، بفعل أمر يصف الأثر:

```
feat(quiz): add data layer
fix(courses): keep the lesson player from freezing the page
refactor: unify feature conventions and document the workflow
chore: remove mock data that the API replaced
```

النوع يقول طبيعة التغيير (ميزة، إصلاح، إعادة هيكلة)، والنطاق يقول أين. فيُقرأ التاريخ بسهولة، ويمكن توليد سجل التغييرات منه آلياً.

س139 لماذا commit بعد كل طبقة بدل commit واحد للميزة؟

متوسط

- **المراجعة أسهل:** فرق الـ domain وحده صغير ومفهوم.
- **التراجع دقيق:** إن كان الخطأ في الـ Cubit أتراجع عن commit واحد.
- **التاريخ يحكي القصة:** domain ثم data ثم cubit ثم الشاشات، بنفس ترتيب المنهج.

وكل commit يجب أن يمر `flutter analyze` و `flutter test` قبله.

س140 كيف كانت مراجعة الكود في مشروعك؟

متوسط

كتبت منهجاً مكتوباً (`docs/FEATURE_WORKFLOW.md`) فيه البنية والأسماء وترتيب الخطوات وقواعد الاعتماد وقائمة الانتهاء. كل فرق يُراجع عليه، وكل ملاحظة تشير إلى القاعدة المكسورة بمكانها، مثل «نص عربي داخل الواجهة» و« `emit` بعد `await` بلا فحص `isClosed` ».

المراجعة شملت أيضاً مقارنة الشاشات بإطارات فيجما، وتجربة الطلبات على الخادم الحقيقي.

س141 ما الفرق بين `merge` و `rebase` ؟

متوسط

- **merge:** يدمج الفرعين بـ commit دمج، ويحفظ التاريخ كما حدث فعلاً، بشكله المتفرع.
- **rebase:** يعيد كتابة commits فرعك فوق آخر `main` ، فيصير التاريخ خطاً مستقيماً نظيفاً.

تجنّب: لا تعِد كتابة تاريخ فرع مشترك دفعته وبنى عليه غيرك (`rebase` أو `push --force`)، فتكسر نسخ الآخرين.

س142 كيف تجهّز نسخة للنشر (release)؟

متوسط

1. **الإصدار** في `pubspec.yaml` : `version: 1.0.0+1` . الجزء الأول `versionName` يراه المستخدم، والرقم بعد `+` `versionCode` يجب أن يزيد مع كل رفع للمتجر.
2. **التوقيع:** مفتاح keystore خاص للنشر، ويُقرأ من `key.properties` . كلاهما خارج Git ومحفوظ بأمان، فضياع المفتاح يمنع تحديث التطبيق نفسه للأبد.
3. **البيئة:** `--dart-define=API_BASE_URL=` لعنوان خادم الإنتاج.
4. **البناء:** `flutter build appbundle` للمتجر، و `--split-per-abi` لملفات APK مباشرة.

في مشروعي: بنيت نسخة release بنجاح. لكنها موقّعة بمفتاح التطوير، وتتصل بخادم التدريب. هاتان الخطوتان المتبقيتان قبل أي نشر حقيقي، وأعرفهما.

س143 كيف تفصل بيئات التطوير والاختبار والإنتاج؟

متقدم

- **بسيط:** `String.fromEnvironment` مع `--dart-define` ، كما في `ApiConfig.baseUrl` .
- **أكمل: Flavors**: نكهة لكل بيئة باسم ومعرّف تطبيق وأيقونة مختلفة (`dev` و `staging` و `prod`)، فتُثبَّت النسخ معاً على جهاز واحد، و `main_dev.dart` و `main_prod.dart` يمرران الإعدادات.

س144 ما الذي يجب ألا يدخل Git أبداً؟

متوسط

- مجلدات البناء: `build/` و `.dart_tool/` .
- الإعدادات المحلية: `android/local.properties` .
- مفاتيح التوقيع (`*.jks` و `key.properties`)، والتوكنات، وكلمات المرور.
- الملفات المؤقتة والشخصية، وملفات ضخمة متغيرة مثل PDF مولّدة. يُحفظ مولّدها لا ناتجها.

تجنّب: سرّ دُفع إلى Git يبقى في التاريخ حتى لو حُذف لاحقاً. الحل تغييره فوراً، لا حذف الملف فقط.

س145 ما دور `flutter analyze` والـ lints؟

متوسط

المحلل يفحص الكود دون تشغيله: أخطاء الأنواع، والاستيرادات غير المستخدمة، والكود الميت، وقواعد الأسلوب من `flutter_lints` . يكشف الأخطاء قبل التشغيل، ويوحّد الأسلوب في الفريق. في مشروعي يجب أن يكون نظيفاً تماماً («No issues found») قبل أي commit.

س146 كيف تحل تعارض الدمج (merge conflict)؟

متوسط

1. أفهم التغييرين: ما أراده كل طرف، لا مجرد اختيار أحدهما.

2. أعدّل الملف ليجمع النيّتين، وأحذف علامات التعارض.
3. أشغّل `flutter analyze` والاختبارات، لأن الدمج الذي يُترجم قد يكسر المنطق.
4. commit للدمج.

وأقلّل التعارضات بالأصل: فروع قصيرة العمر، ودمج متكرر من `main` .

الجزء 13

طريقة التفكير وأسئلة الخبراء

هنا لا يوجد جواب واحد صحيح. السينيور يريد أن يرى كيف تفكر: كيف توازن بين الخيارات، وكيف تعترف بالحدود، وكيف تتعامل مع المجهول.

س147 ما نقاط الضعف المعروفة في مشروعك؟

متقدم

قلها بنفسك وبهدوء. هذا أقوى من أن تُكتشف:

- **التنزيلات ببيانات تجريبية:** حفظ الملفات على الجهاز لم يُبنَ بعد. وفيديوهات يوتيوب لا يمكن تنزيلها أصلاً، فالقرار يحتاج نقاشاً.
- **لا إشعارات فورية:** لا Firebase، والإشعارات تُقرأ عند فتح الصفحة فقط.
- **التوكن غير مشفّر** في `SharedPreferences` .
- **لا عمل بلا إنترنت**، ولا تقسيم صفحات للقوائم، لأن الخادم لا يدعمه.
- **النشر:** النسخة موقّعة بمفتاح التطوير، وتتصل بخادم التدريب.
- **iOS** لم يُبنَ ولم يُجرَّب، لأن العمل كان على Windows.

س148 لو أُعطيت أسبوعين إضافيين، ماذا ستفعل وبأي ترتيب؟

متقدم

بالأولوية حسب الأثر على المستخدم والمخاطرة:

1. **الأمان:** التوكن في التخزين الآمن، والتنسيق مع الخادم لإصلاح تسريب الروابط والإجابات.
2. **جاهزية النشر:** مفتاح التوقيع، وبيئات بـ flavors، وCI يشغّل الفحوص.
3. **المراقبة:** Crashlytics أو Sentry لمعرفة الأعطال عند المستخدمين.
4. **الإشعارات الفورية** بـ Firebase Cloud Messaging.
5. **التنزيلات** لملفات PDF أولاً، فهي ممكنة الآن.

> **تجنّب:** لا تبدأ بـ «سأعيد كتابة كذا بأداة أحدث». ابدأ بما يحمي المستخدم ويجعل التطبيق قابلاً للنشر.

س149 هل استخدمت أدوات الذكاء الاصطناعي؟ وكيف؟

متقدم

كن صادقاً، فالمناقش يعرف أن الجميع يستخدمها. ما يهمه: **هل تفهم ما في مشروعك، وهل تتحمل مسؤوليته؟**

- استخدمتها مساعداً للبرمجة والمراجعة (Cursor وClaude)، مثل زميل يكتب مسودة وأراجعها.
- كتبت القواعد بنفسي في منهج مكتوب (`FEATURE_WORKFLOW.md`)، وكان كل تغيير يُراجع عليه.
- رفضت تغييرات خالفت المنهج: commits مجمّعة، وكود ميت، وأخطاء في مطابقة التصميم.
- اكتشفت بالمراجعة خطأً منطقياً في الاختبارات وأصلحته باختبار.

- التحقق دائماً بأدوات لا تكذب: `flutter analyze`، و338 اختباراً، والتجربة على الخادم الحقيقي.

ثم أثبت الفهم: اعرض أن تشرح أي ملف يختاره المناقش.

تجنّب: لا تنكر الاستخدام، ولا تقل «الذكاء الاصطناعي كتبه». القول الصحيح: «استخدمته أداة، والقرار والفهم والمسؤولية لي».

س150 كيف تتعامل مع مهمة لا تعرف كيف تنفذها؟

متقدم

1. **أفهم المطلوب بدقة:** ما النتيجة المتوقعة؟ وما الحالات الحدية؟
2. **أبحث في المصدر الرسمي أولاً:** توثيق Flutter والحزمة، ثم أمثلة موثوقة.
3. **أجرّب تجربة صغيرة معزولة** قبل إدخالها في المشروع.
4. **أبني على مراحل** بخطوات يمكن التحقق منها، وأكتب اختباراً لما فهمته.
5. **أسأل بعد محاولة حقيقية**، بسؤال محدد: «جرّبت كذا ورأيت كذا».

في مشروعي: مع API غير موثّق: أرسلت طلبات حقيقية، وطلبات فارغة عمداً لأرى قواعد 422، وسجّلت كل ما اكتشفته في تعليقات الـ Models واختبارات البيانات.

س151 كيف تبحث عن خطأ صعب (debugging)؟ احكِ مثالاً.

متقدم

1. **أكرّر الخطأ** بشكل ثابت.
2. **أعزل:** أصغر حالة تُظهره.
3. **أضع فرضية** وأختبرها، لا أغيّر أشياء عشوائياً.
4. **أصلح السبب** لا العرض.
5. **أكتب اختباراً** يمنع رجوعه.

في مشروعي: تجمّد صفحة الدرس: لاحظت أنه يحدث مع مشغّل يوتيوب داخل التمرير فقط. فعزلت القص بالزوايا الدائرية حول الـ WebView، ثم استدعاء تحديث الحالة أثناء بناء المشغّل. وأصلحت السببين: لا قص حول المشغّل، وتأجيل معالجة الخطأ لما بعد الإطار.

س152 التكرار أم الترابط؟ متى تقبل تكرار الكود؟

متقدم

DRY ليس قانوناً مطلقاً. التكرار ثمنه أن التعديل يتم في مكانين، والترابط ثمنه أن تغيير مكان يكسر مكاناً آخر لا علاقة له.

في مشروعي: الرئيسية تقرأ بطاقة الدورة من ردها بدالة تكرر بعض قراءة `CourseModel`، لأن قاعدتي تمنع استيراد طبقة بيانات ميزة أخرى. قبلت عشرين سطراً مكررة مقابل استقلال الميزتين. ولو تكررت في ثلاثة أماكن لنقلتها إلى مكان مشترك مقصود.

القاعدة العملية: تكرار مرتين مقبول، والثالثة إشارة لاستخراج تجريد، بشرط أن يكون التشابه في **المعنى** لا في الشكل فقط.

س153 ماذا تفعل حين يكون الخادم خاطئاً أو غامضاً ولا تملك تعديله؟

متوسط

- **لا أخفي المشكلة بحيلة صامتة** في التطبيق.
- **أحمي المستخدم:** قراءة متسامحة لما لم أره، ورسائل خطأ واضحة، وعدم عرض بيانات مضللة (مثل نسبة إنجاز خاطئة).
- **أوثّق:** تعليق في الـ Model، واختبار بالرد الحقيقي، وقائمة لمطوّر الخادم بكل مشكلة ومثال لها.
- **أتابع:** حين يُصلح الخادم، يكون التعديل في مكان واحد معروف.

س154 كيف يتوسع هذا التطبيق لفريق أكبر ومستخدمين أكثر؟

متقدم

- **الكود:** الميزات منفصلة أصلاً، ويمكن نقلها إلى حزم Dart مستقلة (packages أو melos) بحدود صريحة.
- **الجودة:** CI إلزامي، واختبارات تكامل، وقواعد lint أشد.
- **التشغيل:** مراقبة الأعطال (Crashlytics)، والتحليلات، وميزات تُفعَّل عن بعد (feature flags).
- **البيانات:** تقسيم صفحات، وتخزين محلي، وإعادة محاولة بتأخير متزايد (exponential backoff).

س155 ما الدَّين التقني (technical debt) في مشروعك؟

متقدم

- بعض نصوص المحتوى الثابت (السياسات والأسئلة الشائعة) في ملف محتوى لا في `AppStrings`، لأن الخادم لا يقدمها بعد.
- بعض حالات الشاشات منطقها في الويدجت، مثل `allAnswered` في الـ Cubit بدل الحالة.
- `Failure` ليس `sealed`، وتحليل JSON يدوي.
- الكود غير منسّق بـ `dart format` موحّد.

وأعرف كلفة كل بند، ولا شيء منها يمنع التسليم.

س156 ما الذي يجعل الكود مقروءاً في رأيك؟

متوسط

- **أسماء تقول القصد:** `isAnsweredCorrectly` لا `check`، و`AttendExam` لا `ExamHelper`.
- **دوال وأصناف صغيرة** بمسؤولية واحدة.
- **تعليقات تشرح لماذا لا ماذا:** «401 بلا توكن ليس انتهاء جلسة» أهم من «إذا كان 401».
- **اتساق:** ميزة تشبه الأخرى في البنية والأسماء، فمن فهم واحدة فهم الكل.

س157 الموعد قريب والكود غير مثالي. متى تُعيد الهيكلة ومتى تسلّم؟

متقدم

- أسلّم ما هو **صحيح ومختبر**، وإن لم يكن مثالياً.
- أعيد الهيكلة فوراً إن كان الوضع الحالي **يسبب أخطاء** أو يمنع الميزة التالية.
- وإلا أوثّقه ديناً تقنياً بخطة.

إعادة الهيكلة بلا اختبارات قبل الموعد مخاطرة. ووجود 338 اختباراً هو ما جعل التنظيف الأخير ممكناً بأمان.

س158 لو تباطأ الخادم أو زاد الضغط عليه، ما أول ما سيتأثر في تطبيقك؟

متقدم

- المهلات (15 و20 ثانية) ستظهر خطأ اتصال، ولا إعادة محاولة تلقائية.
- القوائم الكاملة بلا صفحات تكبر مع البيانات، فتصير أبطأ وأثقل على الذاكرة.
- البحث يرسل طلباً بعد كل توقف 400ms، والتأخير يقلل الضغط لكنه لا يلغي الطلب الجاري.

الحلول بالترتيب:

1. تقسيم الصفحات.
2. تخزين محلي يعرض القديم أثناء التحميل.
3. إعادة محاولة بتأخير متزايد.
4. إلغاء الطلب القديم بـ `CancelToken` في Dio.

س159 ما أهم ما تعلمته من هذا المشروع؟

متقدم

اختر شيئاً حقيقياً شعرت به، مثلاً:

- **المعمارية تُختبر عند التغيير لا عند البناء:** ربط الخادم بعد البيانات التجريبية كان تغيير سطر في كل ميزة.
- **البيانات الحقيقية تكشف ما لا يكشفه التوثيق:** أخطاء الخادم، والحقول الغائبة، والأنواع المتغيرة.
- **الاختبارات تمنح الشجاعة:** حذفت مئات الأسطر من الكود الميت قبل التسليم لأن 338 اختباراً تحرسها.

س160 لماذا يجب أن نثق بأن تطبيقك يعمل؟

متوسط

- `flutter analyze` **نظيف** تماماً.
- **338 اختباراً:** لكل ميزة بيانات وCubit وواجهة، واختبار يفتح كل مسار.
- **كل الميزات جُرّبت على الخادم الحقيقي**، والردود الحقيقية محفوظة في الاختبارات.
- **نسخة release تُبنى بنجاح.**
- **الشاشات قورنت بإطارات فيجما** واحدة واحدة.

وأضيف بصراحة ما لم يُختبر: iOS، والأجهزة الحقيقية المتنوعة، والنشر.

مراجعة سريعة: مصطلحات في سطر

اقرأها صباح يوم المناقشة. كل مصطلح بتعريف يمكن قوله في جملة واحدة.

Widget
وصف ثابت لجزء من الواجهة، رخيص ويُعاد إنشاؤه كثيراً.

Element
النسخة الحية للويدجت في الشجرة، تحمل الحالة ودورة الحياة.

RenderObject
يحسب المقاس والموضع ويرسم.

BuildContext
مكان الويدجت في الشجرة، وبه نصل لما فوقها.

Key
هوية للويدجت حين لا يكفي النوع والمكان.

const widget
يُنشأ مرة واحدة، ويتخطاه Flutter عند إعادة البناء.

القيود
تنزل من الأب، والمقاسات تصعد من الابن، والأب يحدد الموضع.

Hot Reload
يحقن الكود ويحفظ الحالة. وHot Restart يعيد من main.

Future / Stream
قيمة واحدة لاحقاً / سلسلة قيم عبر الزمن.

Event loop
خيط واحد يعالج الأحداث، وmicrotasks أولاً.

Isolate
تنفيذ متوازٍ بذاكرة منفصلة، للحساب الثقيل.

Null safety
لا null إلا بإذن صريح (?)، والخطأ يُكتشف وقت الترجمة.

Cubit
دوال تصدر حالات جديدة بـ emit.

Bloc
أحداث تتحول إلى حالات، مع محوّلات توقيت.

BlocBuilder / Listener
يرسم حسب الحالة / ينفّذ أثراً مرة واحدة.

read / watch / select
بلا اشتراك / اشتراك كامل / اشتراك في جزء.

isClosed
لا emit بعد await إن أُغلقت الصفحة.

Equatable
مساواة بالقيمة، فالحالة المتطابقة لا تعيد البناء.

Clean Architecture
domain في القلب، والاعتماد يتجه إلى الداخل.

Entity / Model
بيانات التطبيق / شكل الخادم مع fromJson.

Repository
واجهة في domain، وتنفيذ في data يخفي مصدر البيانات.

UseCase
فعل واحد للتطبيق باسم واضح.

Either
Left للفشل وRight للنجاح، والفشل جزء من النوع.

guard
يحوّل استثناءات المصدر إلى Failure في مكان واحد.

get_it
حقن اعتماديات: LazySingleton للخدمات، وFactory للـ Cubits.

ApiClient
الترويسات والتوكن، وفك غلاف الرد، وتحويل الأخطاء إلى رسائل عربية.

401
انتهاء الجلسة: مسح محلي، وredirect إلى الدخول.

422
أخطاء تحقق: نعرض أول رسالة في errors.

Debounce
انتظار توقف الكتابة 400ms قبل إرسال الطلب.

Stale response
رد قديم يُتجاهل إن لم يطابق النص الحالي.

go vs push
go يعيد بناء المكدّس / push يضيف فوقه.

redirect
حارس مركزي للمسارات حسب الجلسة.

StatefulShellRoute
تبويبات سفلية يحفظ كل منها حالته.

extra
لا يُعتمد عليه: يضيع في الروابط وإعادة البناء.

Fake vs Mock
تنفيذ بسيط يختبر النتيجة / برمجة استدعاءات تختبر التفاصيل.

pump / pumpAndSettle
إطار واحد / حتى تنتهي الحركات.

Golden test
مقارنة صورة الواجهة بمرجع، وهو حساس للخطوط والمنصة.

debug / profile / release
JIT للتطوير / قياس الأداء / AOT للنشر.

Tree shaking
حذف الكود والأيقونات غير المستخدمة من نسخة release.

App Bundle
صيغة المتجر، كل جهاز يأخذ ما يحتاجه.

dart-define

إعداد وقت البناء، مثل عنوان الخادم.

Secure storage

Keychain وKeystore للتوكن بدل SharedPreferences.

الأمان

يُفرض في الخادم، والقفل في التطبيق تجربة استخدام فقط.

RTL

الاتجاه بالبداية والنهاية لا باليمين واليسار، مثل EdgeInsetsDirectional.

Conventional Commits

type(scope): description.

SOLID

مسؤولية واحدة، مفتوح ومغلق، استبدال، فصل واجهات، عكس اعتماد.